Sistema de Ventas para Cafetería — Documentación técnica

Documentación de referencia del proyecto **Sistema de ventas para una cafetería con catálogo e imágenes 3D**, construido sobre Laravel + MariaDB. Este índice describe el stack, la puesta en marcha y el estado de cada módulo; los detalles de datos, autenticación, catálogo 3D y flujo de compra viven en documentos independientes.

Índice

DOCUMENTO	CONTENIDO
01-base-de-datos.md	Diagrama entidad-relación, diccionario de tablas, claves foráneas, relaciones Eloquent y decisiones de diseño del esquema.
02-roles-y-autenticacion.md	Flujo de autenticación con Breeze, sistema de roles, middleware, mapa de rutas protegidas, credenciales semilla y extensión a nuevos roles (RF-01, RNF-03).
03-catalogo-y-visor-3d.md	Catálogo público, ficha de producto, visor Three.js, atributos `data-*`, carga de modelos GLB, manejo de errores y rendimiento (RF-02, RNF-01, RNF-02).
04-carrito-y-pedidos.md	Carrito en sesión, checkout transaccional, validación de stock y precios, métodos de pago, snapshot de importes, checkout de invitado y confirmación firmada (RF-03, RF-04).
05-administracion.md	Panel de administración, CRUD de productos con subida de imagen y modelos .glb/.gltf, gestión de pedidos y cambio de estado (RF-05, RF-06, RNF-03).
06-inventario-y-reportes.md	Inventario de insumos con entradas/salidas y stock automático, reportes de ventas por período y por producto (RF-07, RF-08).

Stack

COMPONENTE	VERSIÓN INSTALADA	NOTAS
PHP	8.4	El proyecto declara `^8.3` en `composer.json`.
Laravel	13.40.0	Incluye atributos `#[Fillable]` y `#[Hidden]` en modelos.
MariaDB	11.8	Base de datos `cafeteria`, charset `utf8mb4`.
Laravel Breeze	2.4.2	Stack Blade + Alpine.js; autenticación con sesiones.
Tailwind CSS	4.x	Integrado mediante `@tailwindcss/vite`.

Three.js	0.187	`three@^0.187.0`, instalado por NPM y empaquetado con Vite (sin CDN); usado por el visor 3D.
Vite	8.x	Bundler de assets (`npm run build` / `npm run dev`).
PHPUnit	12.5	Suite de pruebas (no se usa Pest).

Dependencias relevantes de `composer.json`: `laravel/framework`, `laravel/tinker`, y en desarrollo `laravel/breeze`, `laravel/boost`, `laravel/pint`, `phpunit/phpunit`. Del lado de Node: `three`, `alpinejs`, `tailwindcss`, `vite`, `laravel-vite-plugin`.

El visor 3D requiere un navegador con WebGL. Si WebGL no está disponible, la ficha muestra un panel de error en lugar del modelo, sin afectar al resto del contenido.

Requisitos previos

- PHP 8.3 o superior con las extensiones habituales de Laravel (`pdo_mysql`, `mbstring`, `openssl`, `ctype`, `json`, `bcmath`, `fileinfo`, `tokenizer`, `xml`).
- Composer 2.
- Node.js 20+ y npm.
- MariaDB 11.8 en ejecución.

Puesta en marcha

1. Dependencias de PHP

```
composer install
```

2. Configuración del entorno

```
cp .env.example .env
php artisan key:generate
```

`.env.example` trae `DB_CONNECTION=sqlite`; edita `.env` para apuntar a MariaDB. La configuración verificada del proyecto es:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cafeteria
DB_USERNAME=cafeteria
DB_PASSWORD=cafeteria
```

El proyecto usa `SESSION_DRIVER=database`, `CACHE_STORE=database` y `QUEUE_CONNECTION=database`, por lo que las migraciones crean también las tablas `sessions`, `cache` y `jobs`.

3. Crear la base de datos y el usuario

```
CREATE DATABASE cafeteria CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'cafeteria'@'127.0.0.1' IDENTIFIED BY 'cafeteria';
GRANT ALL PRIVILEGES ON cafeteria.* TO 'cafeteria'@'127.0.0.1';
FLUSH PRIVILEGES;
```

4. Migraciones y datos semilla

```
php artisan migrate --seed
```

El seed es idempotente (`updateOrCreate`) y crea las cuentas descritas en 02-roles-y-autenticacion.md, además del catálogo de ejemplo (Espresso, Latte y Capuchino) descrito en 03-catalogo-y-visor-3d.md:

ROL	EMAIL	CONTRASEÑA
Administrador	`admin@cafeteria.test`	`password`
Cliente	`cliente@cafeteria.test`	`password`

5. Enlace de almacenamiento público

El visor 3D sirve los modelos desde `storage/app/public`, por lo que el symlink `public/storage` debe existir:

```
php artisan storage:link
```

El enlace está en `.gitignore` y debe recrearse en cada entorno. La carpeta `storage/app/public/productos/` no existe hasta que se sube el primer modelo; consulta la guía de carga de modelos.

6. Assets del frontend

```
npm install
npm run build
```

Para desarrollo con recarga en caliente: `npm run dev` o el script combinado `composer run dev`.

7. Servir la aplicación

```
php artisan serve
```

La aplicación queda disponible en `http://127.0.0.1:8000`. La portada `/` es el catálogo público (`productos.index`). El endpoint de salud responde en `/up`.

> *`composer run setup` automatiza instalación, generación de clave, migraciones y build de assets, pero **no** ejecuta los seeders ni crea el symlink de `storage`. Después de usarlo, ejecuta `php artisan db:seed` y `php artisan storage:link`.*

Verificación rápida

```
php artisan route:list          # mapa completo de rutas
php artisan test                # suite PHPUnit
```

Estado verificado de la suite: **81 pruebas aprobadas y 354 aserciones**, incluidas las de autenticación, registro con rol por defecto, acceso al panel de administración, catálogo/visor 3D, carrito, checkout invitado/autenticado, administración de productos y pedidos, e inventario y reportes de ventas.

Para el visor 3D también conviene comprobar manualmente el render en navegador: las pruebas HTTP validan los atributos `data-*`, no la ejecución de WebGL. La lista de comprobación está en 03-catalogo-y-visor-3d.md. El flujo de compra se verifica de extremo a extremo con los pasos de 04-carrito-y-pedidos.md.

Estado de los módulos

MÓDULO	REQUERIMIENTOS	ESTADO ACTUAL	DOCUMENTACIÓN

1. Autenticación y roles	RF-01, RNF-03	**Completado**: registro, login, logout, recuperación de contraseña, rol `cliente` por defecto, middleware `role` y panel `/admin` restringido.	02-roles-y-autenticacion.md
2. Catálogo y visor 3D	RF-02, RNF-01, RNF-02	**Completado**: portada con catálogo paginado de productos activos, ficha de detalle, visor Three.js interactivo sobre `modelo_3d_path`, estados vacío/carga/error, navbar y layout renovados. La subida de modelos aún es manual (Módulo 4).	03-catalogo-y-visor-3d.md
3. Carrito y pedidos	RF-03, RF-04	**Completado**: carrito en sesión con control de stock, checkout transaccional con revalidación y bloqueo de productos, snapshot de precios, métodos de pago, confirmación firmada y checkout de invitado sin cuenta.	04-carrito-y-pedidos.md
4. Administración	RF-05, RF-06, RNF-03	**Completado**: CRUD de productos (create/read/update/delete) con subida de imagen y modelo 3D, gestión de pedidos con cambio de estado, dashboard con métricas reales y panel protegido por rol.	05-administracion.md
5. Inventario y reportes	RF-07, RF-08	**Completado**: CRUD de insumos, registro de movimientos de entrada/salida/ajuste con actualización automática de stock, reportes de ventas por período y por producto, panel protegido por rol.	06-inventario-y-reportes.md

> *El checkout del Módulo 3 admite invitados: no exige autenticación y guarda `pedidos.user_id = NULL`. La confirmación se protege con una URL firmada (`signed`) en lugar de exigir login; la vista es genérica para no mostrar líneas ni datos personales. Los detalles del flujo están en 04-carrito-y-pedidos.md.*

Estructura relevante del proyecto

```
app/
├── Carrito.php             Carrito de compra sobre la sesión (clave `carrito`)
├── Enums/                  Rol, EstadoPedido, MetodoPago, TipoMovimiento
├── Http/
│   ├── Controllers/        Auth (Breeze), Profile, ProductoController, CarritoController,
│   ├── Middleware/         EnsureUserHasRole
│   └── Requests/           LoginRequest, ProfileUpdateRequest, StoreProductoRequest, Updat
├── Models/                 User, Producto, Pedido, DetallePedido, Insumo, MovimientoInvent
└── View/Components/        AppLayout, GuestLayout
bootstrap/app.php           Registro del alias de middleware `role`
database/
├── factories/              Una factory por modelo
├── migrations/             Esquema de datos
└── seeders/                DatabaseSeeder → AdminUserSeeder, ProductoSeeder
resources/
├── css/app.css             Tailwind v4, fuente Instrument Sans
├── js/
│   ├── app.js              Alpine + arranque del visor
│   └── visor-3d.js         Three.js, GLTFLoader, OrbitControls, limpieza GPU
└── views/                  Breeze (auth, profile), productos (index, show), carrito (index
routes/
├── web.php                 Catálogo (/ y /productos/{producto}), carrito, pedidos, panel y
└── auth.php                Rutas de autenticación de Breeze
storage/app/public/         Modelos 3D servidos vía public/storage (vacío en el repo)
tests/                      Feature (Auth, Admin, Profile, ProductoCatalogo, Carrito, Pedid
```

Convenciones de esta documentación

- El idioma es español técnico; los nombres de clases, tablas y rutas se mantienen tal como existen en el código.
- Los diagramas Mermaid se renderizan directamente en visores compatibles (GitHub, GitLab, VS Code).
- Las referencias a requerimientos siguen la nomenclatura del SRS del proyecto: RF para requerimientos funcionales y RNF para no funcionales.

01 — Base de datos

Documento de referencia del esquema relacional del proyecto. Todas las tablas viven en la base MariaDB `cafeteria` y se crean mediante las migraciones de `database/migrations`. El esquema está modelado con Eloquent; los nombres de tabla que no siguen la convención plural de Laravel se declaran explícitamente en el modelo.

- Motor: MariaDB 11.8, charset `utf8mb4`.
- Claves primarias: `bigint unsigned` autoincrementales (`$table->id()`).
- Marcas de tiempo: `created_at` y `updated_at` nullable (`$table->timestamps()`).
- Importes monetarios: `decimal(10,2)`.
- Cantidades de inventario: `decimal(10,3)`.

Diagrama entidad-relación

```
erDiagram
    users |o--o{ pedidos : realiza
    users |o--o{ movimientos_inventario : audita
    pedidos ||--o{ detalle_pedido : contiene
    productos ||--o{ detalle_pedido : referencia
    insumos ||--o{ movimientos_inventario : registra

    users {
        bigint id PK
        varchar name
        varchar email UK
        timestamp email_verified_at
        varchar password
        varchar role
        varchar remember_token
        timestamp created_at
        timestamp updated_at
    }

    productos {
        bigint id PK
        varchar nombre
        text descripcion
        decimal precio
        int stock
        varchar categoria
        varchar imagen_path
        varchar modelo_3d_path
        boolean activo
        timestamp created_at
        timestamp updated_at
    }

    pedidos {
        bigint id PK
        bigint user_id FK
        varchar estado
        varchar metodo_pago
        decimal total
        text notas
        timestamp created_at
        timestamp updated_at
    }

    detalle_pedido {
        bigint id PK
        bigint pedido_id FK
        bigint producto_id FK
        int cantidad
        decimal precio_unitario
        decimal subtotal
        timestamp created_at
        timestamp updated_at
    }

    insumos {
        bigint id PK
        varchar nombre
        varchar unidad_medida
        decimal stock_actual
        decimal stock_minimo
        decimal costo_unitario
        timestamp created_at
```

```
        timestamp updated_at
        }

    movimientos_inventario {
            bigint id PK
        bigint insumo_id FK
            varchar tipo
          decimal cantidad
           varchar motivo
         bigint user_id FK
        timestamp created_at
        timestamp updated_at
        }
```

Cardinalidades del diagrama:

- Un `user` realiza cero o muchos `pedidos`; cada pedido pertenece a cero o un usuario. Los pedidos de invitado tienen `user_id` `NULL`.
- Un `pedido` agrupa cero o muchas líneas de `detalle_pedido`; cada línea pertenece a un solo pedido.
- Un `producto` puede aparecer en cero o muchas líneas de detalle; cada línea referencia un solo producto.
- Un `insumo` acumula cero o muchos `movimientos_inventario`; cada movimiento afecta a un solo insumo.
- Un `user` puede registrar cero o muchos movimientos; `movimientos_inventario.user_id` es opcional, de modo que el histórico sobrevive si se elimina el usuario.

Diccionario de tablas

`users`

Tabla base de Laravel para autenticación, extendida con la columna `role`.

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`name`	`varchar(255)`	NOT NULL	Nombre visible del usuario.
`email`	`varchar(255)`	NOT NULL, UNIQUE	Credencial de acceso.
`email_verified_at`	`timestamp`	NULL	Presente, aunque el modelo `User` no implementa `MustVerifyEmail`.
`password`	`varchar(255)`	NOT NULL	Hash (cast `hashed` en el modelo).
`role`	`varchar(255)`	NOT NULL, default `'cliente'`, INDEX	Añadida por `2026_11_08_160119_add_role_to_users_table` con `after('password')`. Cast al enum `App\Enums\Rol`.
`remember_token`	`varchar(100)`	NULL	Token de "recordarme" de Laravel.
`created_at`	`timestamp`	NULL	
`updated_at`	`timestamp`	NULL	

Índice: `users_role_index` sobre `role`.

El valor de `role` se almacena como cadena y se castea a `Rol` (`Cliente = 'cliente'`, `Admin = 'admin'`). El detalle funcional está en 02-roles-y-autenticacion.md.

`productos`

Catálogo de productos de la cafetería. Los campos `imagen_path` y `modelo_3d_path` preparan el catálogo con visor 3D (RF-02, RNF-01). El uso funcional de `modelo_3d_path` está documentado en 03-catalogo-y-visor-3d.md.

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`nombre`	`varchar(255)`	NOT NULL	
`descripcion`	`text`	NULL	
`precio`	`decimal(10,2)`	NOT NULL	Precio de venta; cast `decimal:2`.
`stock`	`int unsigned`	NOT NULL, default `0`	No admite negativos por el tipo `unsigned`.
`categoria`	`varchar(255)`	NULL	Categoría libre (la factory usa `bebidas`, `postres`, `comida`).
`imagen_path`	`varchar(255)`	NULL	Ruta relativa a `storage/app/public` (p. ej. `productos/latte.jpg`), servida con `asset('storage/...')`.
`modelo_3d_path`	`varchar(255)`	NULL	Ruta relativa a `storage/app/public` (p. ej. `productos/espresso.glb`). Habilita el visor de RF-02.
`activo`	`boolean`	NOT NULL, default `true`	Cast `boolean`; permite ocultar productos sin borrarlos.
`created_at`	`timestamp`	NULL	
`updated_at`	`timestamp`	NULL	

Índice compuesto: `productos_activo_categoria_index` sobre `(activo, categoria)`, pensado para listar el catálogo visible filtrado por categoría.

`pedidos`

Cabecera de cada pedido realizado por un cliente autenticado o por un invitado (RF-03, RF-04).

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`user_id`	`bigint unsigned`	NULL, FK → `users.id`, `RESTRICT`	Autor del pedido; `NULL` en pedidos de invitado.
`estado`	`varchar(255)`	NOT NULL, default `'pendiente'`, INDEX	Cast al enum `App\Enums\EstadoPedido`.
`metodo_pago`	`varchar(255)`	NOT NULL, default `'efectivo'`	Cast al enum `App\Enums\MetodoPago`; añadida por `2026_11_08_163735_add_metodo_pago_and_allow_guest_user_to_pedidos_table`.
`total`	`decimal(10,2)`	NOT NULL, default `0`	Importe total; cast `decimal:2`.
`notas`	`text`	NULL	Observaciones del cliente (máximo 500 caracteres validado en la aplicación).

`created_at`	`timestamp`	NULL	
`updated_at`	`timestamp`	NULL	

Índice: `pedidos_estado_index` sobre `estado`, para filtros operativos por estado.

La migración original declaraba `user_id` como `NOT NULL` con `restrictOnDelete()`. La migración `2026_11_08_163735_add_metodo_pago_and_allow_guest_user_to_pedidos_table` lo hace nullable para permitir el checkout sin cuenta, pero conserva la FK `RESTRICT`: no se puede eliminar un usuario que tenga pedidos. La misma migración agrega `metodo_pago` con default `'efectivo'`.

El `down()` de esa migración es no destructivo: si existen pedidos con `user_id` `NULL`, no revierte los cambios para no perder información y deja un aviso en el log. Por eso no debe asumirse que `migrate:rollback` restaura el esquema original en una base con pedidos de invitado.

`detalle_pedido`

Líneas de producto de cada pedido. El modelo `DetallePedido` fija `protected $table = 'detalle_pedido'` porque Laravel pluralizaría a `detalle_pedidos`.

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`pedido_id`	`bigint unsigned`	NOT NULL, FK → `pedidos.id`, `CASCADE`	Al borrar el pedido se eliminan sus líneas.
`producto_id`	`bigint unsigned`	NOT NULL, FK → `productos.id`, `RESTRICT`	No se puede borrar un producto referenciado.
`cantidad`	`int unsigned`	NOT NULL	Cast `integer`.
`precio_unitario`	`decimal(10,2)`	NOT NULL	Copia del precio al momento de la compra.
`subtotal`	`decimal(10,2)`	NOT NULL	`cantidad × precio_unitario`; cast `decimal:2`.
`created_at`	`timestamp`	NULL	
`updated_at`	`timestamp`	NULL	

La tabla no tiene índice adicional aparte de las claves foráneas; las consultas habituales filtran por `pedido_id`.

`insumos`

Materias primas controladas por inventario (RF-07).

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`nombre`	`varchar(255)`	NOT NULL	
`unidad_medida`	`varchar(255)`	NOT NULL	Texto libre: `gramos`, `mililitros`, `unidades`, etc.
`stock_actual`	`decimal(10,3)`	NOT NULL, default `0`	Cast `decimal:3`.
`stock_minimo`	`decimal(10,3)`	NOT NULL, default `0`	Umbral de reposición; cast `decimal:3`.
`costo_unitario`	`decimal(10,2)`	NULL	Cast `decimal:2`.
`created_at`	`timestamp`	NULL	

`updated_at`	`timestamp`	NULL	

`movimientos_inventario`

Registro auditable de entradas, salidas y ajustes de stock. El modelo `MovimientoInventario` fija `protected $table = 'movimientos_inventario'` porque la convención generaría `movimiento_inventarios`.

COLUMNA	TIPO	RESTRICCIONES	NOTAS
`id`	`bigint unsigned`	PK, autoincremental	
`insumo_id`	`bigint unsigned`	NOT NULL, FK → `insumos.id`, `RESTRICT`	Insumo afectado.
`tipo`	`varchar(255)`	NOT NULL	Cast al enum `App\Enums\TipoMovimiento`.
`cantidad`	`decimal(10,3)`	NOT NULL	Magnitud del movimiento; cast `decimal:3`.
`motivo`	`varchar(255)`	NULL	Justificación opcional.
`user_id`	`bigint unsigned`	NULL, FK → `users.id`, `SET NULL`	Responsable del registro; se conserva el movimiento si el usuario se elimina.
`created_at`	`timestamp`	NULL	
`updated_at`	`timestamp`	NULL	

Índice compuesto: `movimientos_inventario_insumo_id_created_at_index` sobre `(insumo_id, created_at)`, útil para reconstruir el historial de un insumo en orden cronológico.

Enumeraciones de dominio

Los cuatro enums son `string` y se guardan como texto en la base de datos. No existen restricciones `CHECK` en MariaDB: la validación del dominio ocurre en PHP mediante el cast de Eloquent y, en el checkout, mediante reglas `in:`.

ENUM	VALORES	TABLA / COLUMNA
`App\Enums\Rol`	`cliente`, `admin`	`users.role`
`App\Enums\EstadoPedido`	`pendiente`, `en_preparacion`, `listo`, `entregado`, `cancelado`	`pedidos.estado`
`App\Enums\MetodoPago`	`efectivo`, `tarjeta`, `transferencia`	`pedidos.metodo_pago`
`App\Enums\TipoMovimiento`	`entrada`, `salida`, `ajuste`	`movimientos_inventario.tipo`

Relaciones Eloquent

MODELO	MÉTODO	TIPO	MODELO DESTINO	FK USADA
`User`	`pedidos()`	`hasMany`	`Pedido`	`pedidos.user_id`
`User`	`movimientosInventario()`	`hasMany`	`MovimientoInventario`	`movimientos_inventario.user_id`
`Producto`	`detallesPedido()`	`hasMany`	`DetallePedido`	`detalle_pedido.producto_id`
`Pedido`	`user()`	`belongsTo`	`User`	`pedidos.user_id`

`Pedido`	`detallesPedido()`	`hasMany`	`DetallePedido`	`detalle_pedido.pedido_id`
`DetallePedido`	`pedido()`	`belongsTo`	`Pedido`	`detalle_pedido.pedido_id`
`DetallePedido`	`producto()`	`belongsTo`	`Producto`	`detalle_pedido.producto_id`
`Insumo`	`movimientosInventario()`	`hasMany`	`MovimientoInventario`	`movimientos_inventario.insumo_id`
`MovimientoInventario`	`insumo()`	`belongsTo`	`Insumo`	`movimientos_inventario.insumo_id`
`MovimientoInventario`	`user()`	`belongsTo`	`User`	`movimientos_inventario.user_id`

Los modelos usan el atributo `#[Fillable([...])]` de Laravel 13 en lugar de la propiedad `$fillable`. Todos cuentan con factory en `database/factories`, lo que permite generar datos de prueba coherentes (por ejemplo, `DetallePedidoFactory` calcula `subtotal` a partir de `cantidad` y `precio_unitario`).

`Pedido::user()` es una relación `belongsTo` opcional: devuelve `null` en pedidos de invitado. A la inversa, `User::pedidos()` solo incluye pedidos con `user_id`; los pedidos anónimos no aparecen en el historial de ningún usuario.

Decisiones de diseño

Importes en `decimal`, nunca en `float`

`precio`, `total`, `precio_unitario`, `subtotal` y `costo_unitario` usan `decimal(10,2)`. Evita errores de redondeo binario en operaciones monetarias y permite sumar totales con precisión de centavos. El cast `decimal:2` devuelve cadenas con dos decimales, aptas para presentación.

Cantidades de inventario con tres decimales

`stock_actual`, `stock_minimo` y `movimientos_inventario.cantidad` usan `decimal(10,3)` porque los insumos se miden en gramos o mililitros y necesitan fracciones. El stock de productos terminados (`productos.stock`) es entero sin signo, ya que se cuenta en unidades.

Borrado en cascada solo donde no hay pérdida de información

- `detalle_pedido.pedido_id` es `CASCADE`: las líneas no tienen sentido sin su pedido y se eliminan con él.
- `pedidos.user_id`, `detalle_pedido.producto_id` y `movimientos_inventario.insumo_id` son `RESTRICT`: impiden borrar entidades con historial asociado y protegen la integridad de pedidos e inventario.
- `movimientos_inventario.user_id` es `SET NULL`: el movimiento es un registro de auditoría y debe sobrevivir aunque se elimine la cuenta del responsable.

Checkout de invitado con `user_id` nullable

El flujo de compra no exige cuenta: `pedidos.user_id` admite `NULL` para los pedidos de invitado. La FK contra `users.id` se conserva con `RESTRICT`, de modo que la integridad histórica no cambia para los pedidos autenticados y no se puede borrar una cuenta con pedidos. El acceso a la confirmación no se apoya en `user_id`, sino en una URL firmada (`signed`); por eso la vista es genérica y no muestra datos personales. Los detalles funcionales están en 04-carrito-y-pedidos.md.

La contrapartida es que los pedidos anónimos no pueden asociarse a un cliente ni listarse desde `User::pedidos()`. Si en el futuro se exige historial de invitados, habría que añadir un identificador de sesión o un token de consulta, no reutilizar `user_id` con un valor ficticio.

`metodo_pago` como enum de texto

`pedidos.metodo_pago` es `varchar(255)` con default `'efectivo'`, casteado a `App\Enums\MetodoPago`. Al igual que `estado` y `role`, no hay `CHECK` en la base: la aplicación valida el valor con `in:efectivo,tarjeta,transferencia` y el cast falla si se hidrata un valor desconocido. El default de la columna solo cubre inserciones directas por SQL; el checkout siempre escribe un valor validado.

Instantáneas de precio en las líneas de pedido

`detalle_pedido` guarda `precio_unitario` y `subtotal` en lugar de calcularlos siempre contra `productos.precio`. Así, un cambio de precio posterior no altera el importe histórico del pedido. `pedidos.total` almacena el total consolidado.

`modelo_3d_path` como ruta, no como binario

Tanto `imagen_path` como `modelo_3d_path` son `varchar(255)` nullable que almacenan rutas **relativas a `storage/app/public`**, no binarios. El archivo vive en el disco `public` (`storage/app/public/productos/*.glb`) y se expone mediante el symlink `public/storage` creado con `php artisan storage:link`; Blade construye la URL con `asset('storage/'.$producto->modelo_3d_path)`. Por eso la ruta guardada no debe incluir el prefijo `storage/` ni una barra inicial.

Guardar el archivo fuera de la base de datos mantiene las consultas ligeras y permite servirlo como estático con caché; es la base del visor interactivo de RF-02. Se recomienda GLB binario con texturas incrustadas y sin compresión Draco (el visor no registra `DRACOLoader`), en línea con el objetivo de carga de RNF-02. Las factories generan ambos campos como `null`, y `ProductoSeeder` también: el repositorio no incluye modelos de ejemplo. La asignación de rutas es manual hasta que el Módulo 4 incorpore la subida desde el panel; la guía operativa está en 03-catalogo-y-visor-3d.md.

Índices orientados a las consultas previstas

ÍNDICE	TABLA	USO ESPERADO
`(activo, categoria)`	`productos`	Catálogo visible agrupado/filtrado por categoría.
`estado`	`pedidos`	Bandeja de pedidos por estado.
`role`	`users`	Filtrado de usuarios administradores.
`(insumo_id, created_at)`	`movimientos_inventario`	Historial cronológico por insumo.

Nombres de tabla explícitos

`DetallePedido` y `MovimientoInventario` declaran `$table` porque la pluralización automática de Eloquent no coincide con los nombres migrados (`detalle_pedido` y `movimientos_inventario`). Cualquier consulta con Query Builder debe usar esos nombres exactos.

Valores por defecto en dos capas

`users.role` y `pedidos.estado` tienen default tanto en la migración como en el modelo/enum. Esto mantiene la coherencia cuando se crean registros por SQL directo o por Eloquent. En el caso de `User`, además, `$attributes = ['role' => 'cliente']` garantiza el rol incluso antes de guardar.

Tablas de infraestructura de Laravel

Además de las tablas de negocio, las migraciones base crean:

TABLA	PROPÓSITO
`password_reset_tokens`	Tokens de recuperación de contraseña de Breeze.
`sessions`	Sesiones (el proyecto usa `SESSION_DRIVER=database`).
`cache`, `cache_locks`	Caché y bloqueos (`CACHE_STORE=database`).
`jobs`, `job_batches`, `failed_jobs`	Cola de trabajos (`QUEUE_CONNECTION=database`).

Operaciones útiles

```
# Recrear el esquema y cargar las cuentas semilla
php artisan migrate:fresh --seed

# Inspeccionar la definición real de una tabla en MariaDB
mysql -u cafeteria -p cafeteria -e "SHOW CREATE TABLE productos\G"
```

No se usan *soft deletes* ni restricciones `CHECK`; la integridad de dominio se apoya en enums PHP, tipos de columna y claves foráneas. Para el comportamiento de autenticación y autorización sobre `users.role`, consulta 02-roles-y-autenticacion.md. Para el uso real de `productos.modelo_3d_path`, el visor Three.js y la carga de archivos GLB, consulta 03-catalogo-y-visor-3d.md. Para el carrito, el checkout transaccional y la confirmación de pedidos, consulta 04-carrito-y-pedidos.md.

02 — Roles y autenticación

Este documento describe el módulo de autenticación y control de acceso, que cubre **RF-01** (registro e inicio de sesión de clientes) y **RNF-03** (autenticación con roles cliente/admin para restringir el panel administrativo). La implementación se apoya en Laravel Breeze 2.4.2 con stack Blade y sesiones de Laravel.

Componentes del módulo

COMPONENTE	ARCHIVO	RESPONSABILIDAD
Rutas de autenticación	`routes/auth.php`	Registro, login, logout, recuperación y confirmación de contraseña.
Rutas principales	`routes/web.php`	Catálogo público, dashboard, perfil y grupo `/admin` protegido.
Registro	`app/Http/Controllers/Auth/RegisteredUserController.php`	Crea usuarios con rol por defecto `cliente`.
Login	`app/Http/Controllers/Auth/AuthenticatedSessionController.php` + `app/Http/Requests/Auth/LoginRequest.php`	Autenticación con rate limiting y regeneración de sesión.
Enum de roles	`app/Enums/Rol.php`	Valores `cliente` y `admin`.
Modelo	`app/Models/User.php`	Cast de `role`, helpers `isAdmin()` / `isCliente()` y relaciones.
Middleware	`app/Http/Middleware/EnsureUserHasRole.php`	Restringe rutas por rol.
Registro del alias	`bootstrap/app.php`	Alias `role` → `EnsureUserHasRole`.
Panel administrativo	`app/Http/Controllers/AdminDashboardController.php` + `resources/views/admin/dashboard.blade.php`	Vista inicial del panel.
Seeder	`database/seeders/AdminUserSeeder.php`	Cuentas de administrador y cliente de ejemplo.

Definición de roles

El rol se modela con un enum `string` respaldado por la columna `users.role` (`varchar(255)`, default `'cliente'`, indexada).

```php
enum Rol: string
{
    case Cliente = 'cliente';
    case Admin = 'admin';
}
```

CASO	VALOR PERSISTIDO	DESCRIPCIÓN
`Rol::Cliente`	`cliente`	Usuario final: consulta catálogo y realiza pedidos.
`Rol::Admin`	`admin`	Acceso al panel de administración.

El modelo aplica el cast y concentra la comprobación de roles:

```php
protected function casts(): array
{
    return [
        'role' => Rol::class,
        // ...
    ];
}

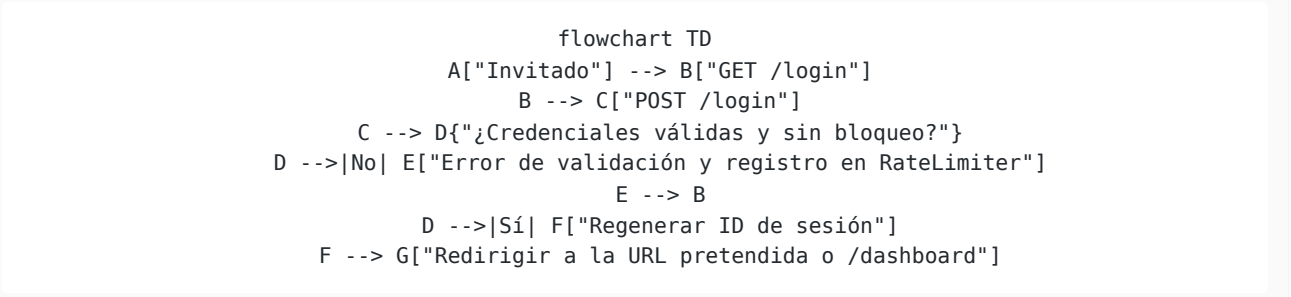
public function isAdmin(): bool
{
    return $this->role === Rol::Admin;
}
```

Puntos clave:

- `role` está incluido en `#[Fillable(['name', 'email', 'password', 'role'])]`, pero el registro público **no** lo asigna: `RegisteredUserController` solo envía `name`, `email` y `password`. El valor `cliente` proviene del default de la migración y de `$attributes = ['role' => 'cliente']` en el modelo.
- El acceso a `$user->role` devuelve una instancia de `Rol`, no una cadena.
- Los helpers `isAdmin()` e `isCliente()` se usan en las vistas para decidir qué enlaces mostrar.
- El rol es único por usuario: no existe una tabla pivote de roles ni permisos granulares.

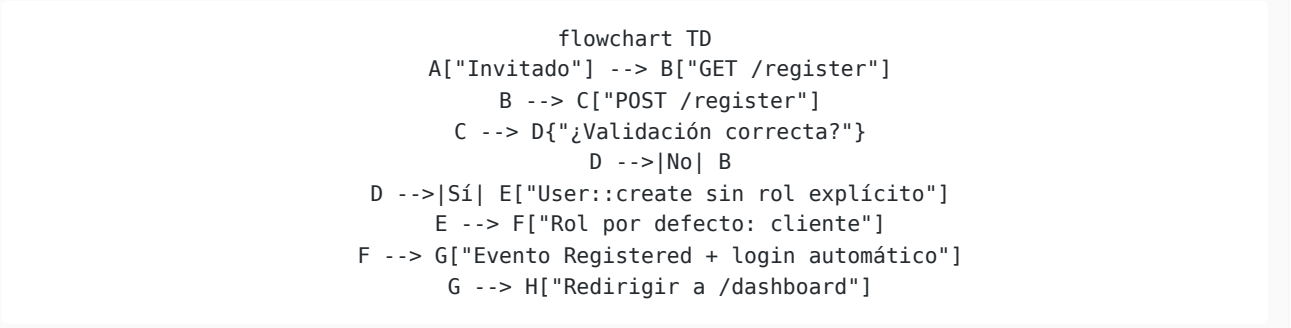
Flujo de autenticación

Inicio de sesión



- `LoginRequest` valida `email` y `password` y limita a **5 intentos** por combinación de email + IP; al superarlos se dispara el evento `Lockout`.
- `Auth::attempt()` verifica las credenciales; la contraseña se almacena con cast `hashed`.
- Tras autenticar, la sesión se regenera para prevenir fijación de sesión y se redirige con `intended()`.

Registro

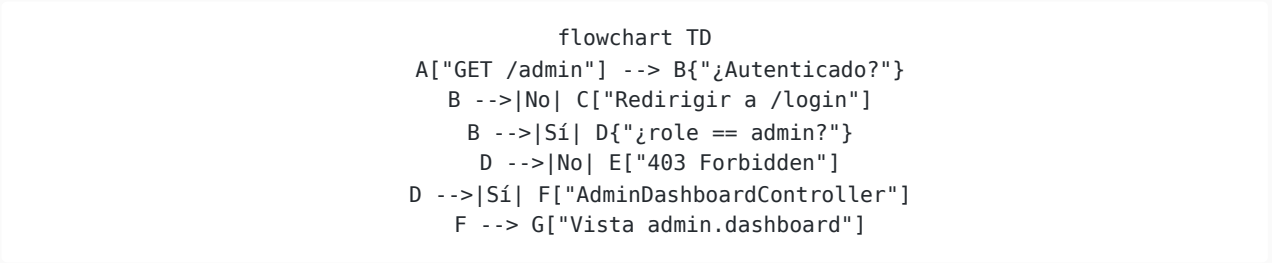


La validación exige `name`, `email` único y `password` confirmado con las reglas por defecto de Laravel.

Cierre de sesión

`POST /logout` ejecuta `Auth::guard('web')->logout()`, invalida la sesión, regenera el token CSRF y redirige a `/`.

Acceso al panel administrativo



Middleware de rol

`EnsureUserHasRole` recibe uno o varios roles permitidos como parámetros del middleware:

```php
public function handle(Request $request, Closure $next, string ...$roles): Response
{
    $role = $request->user()?->role;

    if ($role === null || ! in_array($role->value, $roles, true)) {
        abort(403);
    }

    return $next($request);
}
```

El alias se registra en `bootstrap/app.php`:

```php
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        'role' => EnsureUserHasRole::class,
    ]);
})
```

Comportamiento:

- Invitado: `$request->user()` es `null`, pero el middleware `auth` se ejecuta antes y redirige a `/login`; `role` nunca llega a evaluarse.
- Usuario autenticado con rol distinto: `abort(403)`.
- Comparación estricta contra `$role->value`, por lo que `role:admin` se escribe con el valor del enum, no con el nombre del caso (`Admin`).
- Al ser variádico, admite listas: `role:admin,empleado`.

Mapa de rutas y protección

Rutas públicas

MÉTODO	URI	NOMBRE	MIDDLEWARE	ACCIÓN
GET	`/`	`productos.index`	web	Catálogo público (`ProductoController@index`); sustituye a la vista `welcome` .
GET	`/productos/{producto}`	`productos.show`	web	Ficha y visor 3D (`ProductoController@show`); 404 si el producto está inactivo.
GET	`/carrito`	`carrito.index`	web	Carrito de compra (`CarritoController@index`).
POST	`/carrito/agregar`	`carrito.add`	web	Agregar unidades con validación de stock (`CarritoController@add`).
POST	`/carrito/quitar`	`carrito.remove`	web	Restar unidades o eliminar la línea (`CarritoController@remove`).
POST	`/carrito/vaciar`	`carrito.clear`	web	Vaciar el carrito (`CarritoController@clear`).
POST	`/pedido/confirmar`	`pedidos.store`	web	Checkout transaccional (`PedidoController@store`).
GET	`/pedido/{pedido}/confirmado`	`pedidos.confirmado`	web, `signed`	Confirmación con URL firmada (`PedidoController@confirmado`).
GET	`/up`	—	—	Health check de Laravel.

Las rutas del catálogo, el carrito y los pedidos son públicas y no interfieren con la autenticación: un invitado puede navegar el catálogo y confirmar un pedido sin iniciar sesión. El carrito vive en la sesión; no se exige una cuenta. La confirmación no usa el middleware `auth` , sino `signed` : la URL firmada es la que habilita consultar número, total y método de pago. Los detalles del catálogo están en 03-catalogo-y-visor-3d.md; los del flujo de compra, en 04-carrito-y-pedidos.md.

Rutas para invitados (`guest`)

MÉTODO	URI	NOMBRE	ACCIÓN
GET	`/register`	`register`	Formulario de registro.
POST	`/register`	—	Crear usuario y autenticarlo.
GET	`/login`	`login`	Formulario de login.
POST	`/login`	—	Autenticar credenciales.

GET	`/forgot-password`	`password.request`	Solicitar enlace de recuperación.
POST	`/forgot-password`	`password.email`	Enviar enlace por correo.
GET	`/reset-password/{token}`	`password.reset`	Formulario de nueva contraseña.
POST	`/reset-password`	`password.store`	Guardar la nueva contraseña.

Rutas autenticadas (`auth`)

MÉTODO	URI	NOMBRE	MIDDLEWARE ADICIONAL	ACCIÓN
GET	`/dashboard`	`dashboard`	—	Vista `dashboard`.
GET	`/profile`	`profile.edit`	—	Editar perfil.
PATCH	`/profile`	`profile.update`	—	Actualizar datos.
DELETE	`/profile`	`profile.destroy`	—	Eliminar cuenta.
GET	`/verify-email`	`verification.notice`	—	Aviso de verificación.
GET	`/verify-email/{id}/{hash}`	`verification.verify`	`signed`, `throttle:6,1`	Verificar correo.
POST	`/email/verification-notification`	`verification.send`	`throttle:6,1`	Reenviar verificación.
GET	`/confirm-password`	`password.confirm`	—	Confirmar contraseña.
POST	`/confirm-password`	—	—	Validar contraseña.
PUT	`/password`	`password.update`	—	Cambiar contraseña.
POST	`/logout`	`logout`	—	Cerrar sesión.

> *Las rutas de verificación de correo existen porque Breeze las publica, pero `App\Models\User` **no** implementa `MustVerifyEmail` (la importación está comentada). Por tanto, la verificación no se exige para acceder al dashboard.*

Rutas de administración

MÉTODO	URI	NOMBRE	MIDDLEWARE	ACCIÓN
GET	`/admin`	`admin.dashboard`	`auth`, `role:admin`	`AdminDashboardController` (invokable) → vista `admin.dashboard`.

Definición en `routes/web.php`:

```php
Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::get('/', AdminDashboardController::class)->name('dashboard');
    });
```

La protección es doble:

1. **Servidor**: el middleware impide el acceso directo; un cliente autenticado recibe `403`.
2. **Interfaz**: el enlace "Administración" de `layouts/navigation.blade.php` se muestra solo dentro de `@if (Auth::user()->isAdmin())`, tanto en la navegación de escritorio como en la móvil. Ocultar el enlace es solo cosmético; la autorización real está en la ruta. La misma navbar incluye "Catálogo" para todos y "Panel" para usuarios autenticados.

Credenciales semilla

`DatabaseSeeder` invoca `AdminUserSeeder`, que usa `updateOrCreate` por email (idempotente):

ROL	NOMBRE	EMAIL	CONTRASEÑA
`admin`	Administrador Cafetería	`admin@cafeteria.test`	`password`
`cliente`	Cliente de Ejemplo	`cliente@cafeteria.test`	`password`

```bash
# Ejecuta todos los seeders registrados
php artisan db:seed

# Solo las cuentas de ejemplo
php artisan db:seed --class=AdminUserSeeder
```

> *El seeder actualiza las cuentas existentes: volver a ejecutarlo restablece la contraseña a `password` y fuerza el rol definido. No lo uses en producción con cuentas reales.*

Cómo crear un administrador

Opción A — cuenta semilla (entorno local):

```bash
php artisan db:seed --class=AdminUserSeeder
```

Opción B — promover un usuario existente:

```
php artisan tinker --execute 'App\Models\User::where("email", "nuevo@cafeteria.test")->update(["r
```

Opción C — en pruebas automatizadas, usando el estado de la factory:

```
$admin = User::factory()->admin()->create();
$cliente = User::factory()->cliente()->create();
```

Para revocar el acceso, asigna `Rol::Cliente->value` con el mismo patrón.

Cómo extender el sistema a más roles

El rol es una columna `varchar` casteada a enum, por lo que añadir un valor no requiere cambiar el tipo de dato:

1. **Añadir el caso al enum** `app/Enums/Rol.php`:

   ```
   case Empleado = 'empleado';
   ```

2. **Actualizar la lógica de dominio**: añadir un helper si se necesita (`isEmpleado()`), ajustar factories (`UserFactory`), seeders y vistas que ramifiquen por rol.
3. **Proteger las rutas** con el valor persistido:

   ```
   Route::middleware(['auth', 'role:admin,empleado'])->prefix('admin')->group(...);
   ```

4. **Migración**: no es necesaria para aceptar el nuevo valor. Solo si cambia el rol por defecto o se renombra un valor existente; en ese caso, migra primero los datos para evitar que el cast `Rol::from()` falle al hidratar registros con valores desconocidos.
5. **Pruebas**: añadir casos de acceso permitido/denegado siguiendo `tests/Feature/Admin/AdminDashboardTest.php` .

Consideraciones:

- El middleware compara cadenas contra `$role->value` ; `role:Empleado` no funcionaría, debe usarse `role:empleado` .
- Para permisos por recurso o por acción (por ejemplo, "este empleado solo edita productos"), el rol único se queda corto: usa `Policy` / `Gate` de Laravel y reserva los roles para la separación gruesa cliente/admin.
- Si en el futuro se necesitan múltiples roles por usuario, habría que migrar a una tabla pivote; el esquema actual guarda un solo valor.

Pruebas del módulo

```
# Suite completa
php artisan test

# Solo el panel administrativo
php artisan test --filter=AdminDashboardTest

# Solo registro
php artisan test --filter=RegistrationTest
```

Cobertura relevante verificada:

PRUEBA	ESCENARIO	RESULTADO ESPERADO
`AdminDashboardTest::test_guest_is_redirected_to_login`	Invitado solicita `/admin`.	Redirección a `login`.
`AdminDashboardTest::test_customer_receives_403`	Cliente autenticado solicita `/admin`.	`403 Forbidden`.
`AdminDashboardTest::test_admin_can_access_the_dashboard`	Administrador solicita `/admin`.	`200` y texto "Panel de administración".
`RegistrationTest::test_new_users_can_register`	Registro público.	Usuario autenticado y fila con `role = 'cliente'`.

Estado global de la suite: **48 pruebas aprobadas y 172 aserciones** (incluye las 5 pruebas de `ProductoCatalogoTest` y las 15 de `CarritoTest` / `PedidoTest`).

Notas de seguridad

- El registro público no acepta `role`; no incorpores ese campo a la validación de `RegisteredUserController` sin una comprobación de autorización explícita.
- Ante acceso denegado se devuelve `403`, no `404`: la existencia del panel no se oculta, pero su contenido queda protegido.
- El login limita intentos y el logout invalida la sesión y regenera el token CSRF.
- Las contraseñas se almacenan con hash mediante el cast `hashed`.
- El middleware `role` debe ejecutarse siempre después de `auth` para que un invitado reciba una redirección al login en lugar de un `403`.
- La verificación de correo está disponible a nivel de rutas, pero no se aplica porque el modelo no implementa `MustVerifyEmail`.

Referencia a RNF-03

EXIGENCIA	IMPLEMENTACIÓN
Autenticación de usuarios	Breeze con guard `web` y sesiones en base de datos.
Distinción cliente/admin	Enum `Rol`, columna `users.role` indexada y cast en el modelo.
Restricción del panel	Grupo `/admin` con middleware `['auth', 'role:admin']` y `abort(403)` para roles no autorizados.
Alta controlada de administradores	Seeder idempotente y promoción explícita por consola; el registro público crea solo clientes.
Verificación del comportamiento	Pruebas feature para invitado, cliente y administrador.

Para el esquema de la columna `role` y el resto de tablas, consulta 01-base-de-datos.md. Para el catálogo público y el visor 3D, consulta 03-catalogo-y-visor-3d.md. Para el carrito y el checkout —incluido el flujo sin login—, consulta 04-carrito-y-pedidos.md. Para la puesta en marcha del proyecto, vuelve al índice.

03 — Catálogo y visor 3D

Este documento describe el módulo público de catálogo y el visor interactivo de modelos 3D. Cubre **RF-02** (catálogo de productos con vista tridimensional), **RNF-01** (visor 3D interactivo y usable) y **RNF-02** (carga ágil de la ficha y del modelo, con objetivo de menos de 3 segundos).

El módulo sustituye la portada por el catálogo real: `GET /` ya no renderiza `welcome`, sino `productos.index`. La autenticación y el panel administrativo descritos en 02-roles-y-autenticacion.md siguen intactos.

Alcance y requerimientos

REQUERIMIENTO	IMPLEMENTACIÓN ACTUAL
RF-02 — catálogo con imágenes/modelos 3D	Listado público de productos activos, ficha de detalle y visor Three.js para los productos que tengan `modelo_3d_path`.
RNF-01 — visor interactivo	Rotación con arrastre, zoom con rueda o pinza, encuadre automático, auto-rotación suave y estados alternativos accesibles.
RNF-02 — rendimiento	GLB como formato recomendado, servido como archivo estático desde `public/storage`, límite de *device pixel ratio* y liberación de recursos GPU.

Fuera de alcance de este módulo: el CRUD administrativo de productos y la subida de archivos desde la interfaz, que se implementaron en el Módulo 4 y se documentan en 05-administracion.md. El carrito y el checkout se implementaron en el Módulo 3; el botón "Agregar al carrito" de la ficha ya envía un formulario real a `carrito.add`, documentado en 04-carrito-y-pedidos.md.

Componentes del módulo

COMPONENTE	ARCHIVO	RESPONSABILIDAD
Catálogo	`app/Http/Controllers/ProductoController.php` (`index`) + `resources/views/productos/index.blade.php`	Lista productos activos con paginación de 9 y estado vacío.
Ficha de producto	`ProductoController::show` + `resources/views/productos/show.blade.php`	Detalle, estado de stock y contenedor del visor 3D.
Visor	`resources/js/visor-3d.js`	Inicializa Three.js sobre `[data-visor-3d]`, carga el GLB, gestiona cámara, luces, controles, errores y limpieza.

Punto de entrada JS	`resources/js/app.js`	Arranca Alpine y llama a `inicializarVisores3D()` al terminar el parseo del DOM.
Rutas	`routes/web.php`	`productos.index` y `productos.show`.
Datos semilla	`database/seeders/ProductoSeeder.php`	Tres cafés de ejemplo, sin imagen ni modelo 3D.
Pruebas	`tests/Feature/ProductoCatalogoTest.php`	Cobertura HTTP del catálogo, detalle, 404 y atributos del visor.
Navegación	`resources/views/layouts/navigation.blade.php`	Navbar sticky con enlace "Catálogo" y accesos según sesión/rol.

`three@^0.187.0` se instala por NPM y se empaqueta con Vite; no se usa CDN. El visor importa `three` y, desde `three/addons`, `OrbitControls` y `GLTFLoader`.

Arquitectura Blade → atributos `data-*` → JavaScript

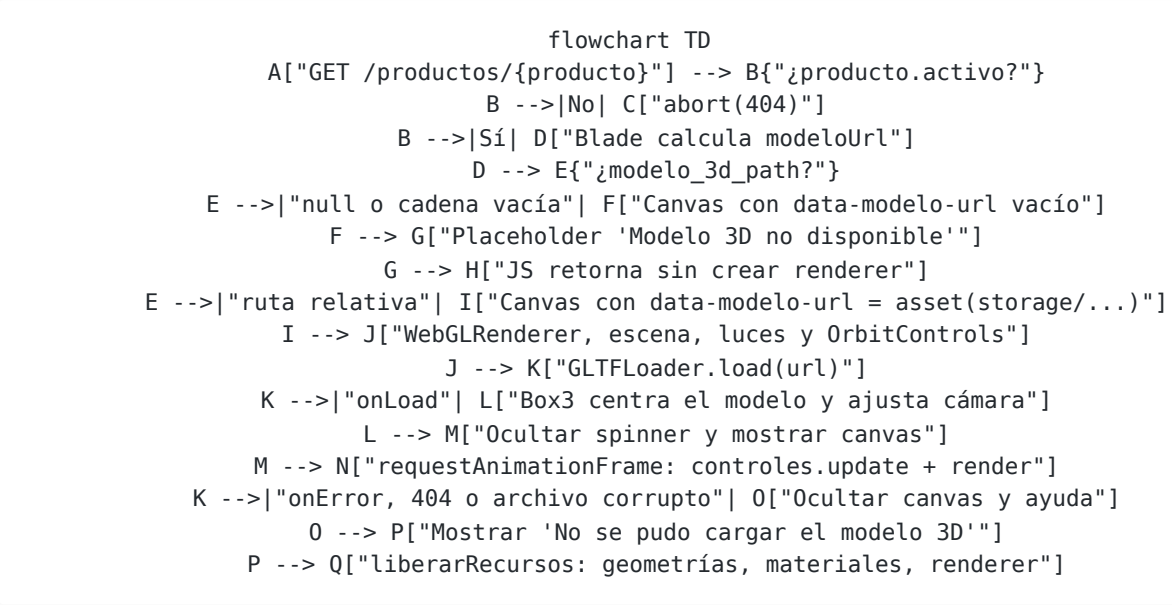
La vista no contiene lógica de Three.js. Blade calcula la URL pública del modelo y la publica como atributos `data-*`; el script descubre los canvas y decide si debe renderizar.

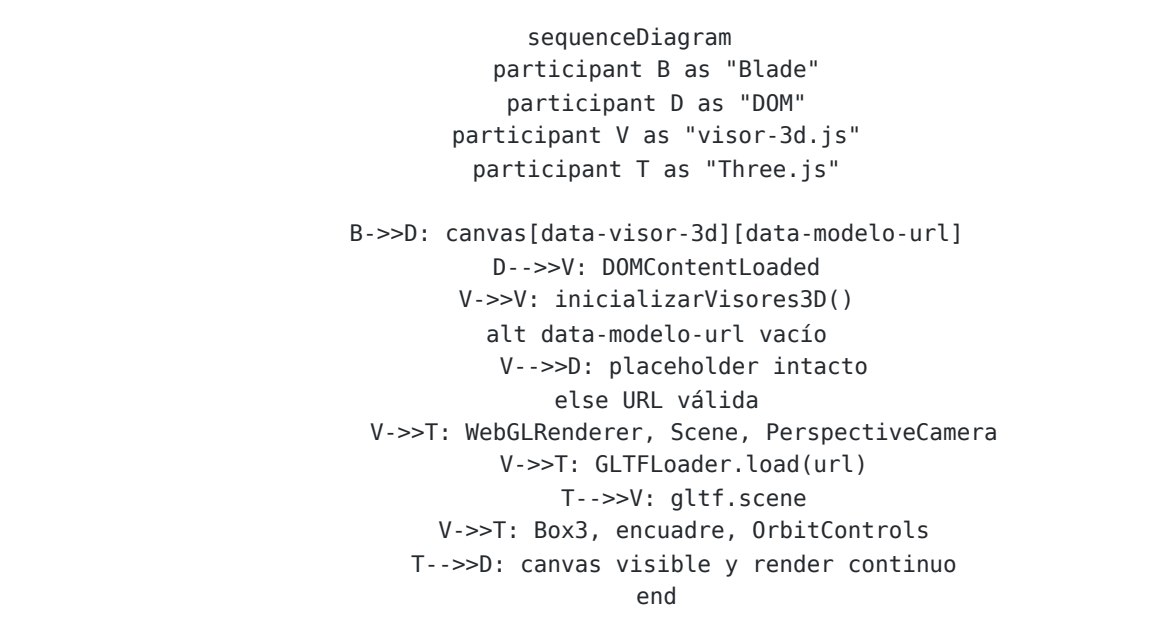
```
ProductoController::show
        │
        ▼
show.blade.php  ── calcula $modeloUrl = asset('storage/'.$producto->modelo_3d_path)
        │
        ▼
<canvas data-visor-3d data-modelo-url="…"> + estados HTML (spinner, placeholder, error)
        │
        ▼
app.js → inicializarVisores3D() → inicializarVisor(canvas)
        │
        ├── URL vacía → no se crea renderer; queda el placeholder del servidor
        └── URL válida → WebGLRenderer + GLTFLoader + OrbitControls + render loop
```

Consecuencias de este diseño:

- El HTML es útil sin JavaScript: el nombre, precio, stock y descripción están en la columna de texto y, si no hay modelo, el placeholder también se renderiza en el servidor.
- Un producto sin modelo no cuesta recursos: el script sale antes de construir el renderer.
- La URL se genera con `asset()`, por lo que depende de `APP_URL` y del symlink `public/storage`.
- El canvas lleva `data-modelo-alt`, pero el nombre accesible se resuelve con `aria-label` generado en Blade; el atributo queda como dato informativo.

Flujo de render





Rutas y controlador

MÉTODO	URI	NOMBRE	MIDDLEWARE	ACCIÓN
GET	`/`	`productos.index`	web	`ProductoController@index`

GET	`/productos/{producto}`	`productos.show`	web	`ProductoController@show` con *route model binding*

Definición en `routes/web.php`:

```
Route::get('/', [ProductoController::class, 'index'])->name('productos.index');
Route::get('/productos/{producto}', [ProductoController::class, 'show'])->name('productos.s
```

Comportamiento del controlador:

- `index()` filtra `where('activo', true)`, ordena por `categoria` y luego `nombre`, y pagina de 9 en 9 con `withQueryString()` para conservar query params en los enlaces de paginación. El índice compuesto `(activo, categoria)` respalda el filtro.
- `show()` recibe el `Producto` por binding implícito y ejecuta `abort_unless($producto->activo, 404)`. Un producto inactivo no se oculta con 403: simplemente no existe para el visitante.
- Un `id` inexistente también produce 404 mediante el binding.
- Las rutas son públicas; no requieren sesión.

Las dos vistas

Catálogo (`resources/views/productos/index.blade.php`)

- Contenedor `mx-auto max-w-7xl px-4 sm:px-6 lg:px-8` con cabecera editorial en tonos stone y ámbar.
- Grid responsivo: `grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`, separación `gap-8`.
- Tarjeta: `rounded-2xl border border-stone-200 bg-white shadow-sm`, transición a `hover:-translate-y-0.5 hover:shadow-lg`.
- Imagen: relación `aspect-[4/3]`; si `imagen_path` existe se usa `object-cover` con `loading="lazy"`; si no, se pinta un degradado y un icono SVG de taza.
- Badge de categoría en la esquina superior izquierda y estado de stock ("Disponible"/"Agotado") en la derecha, ambos con `backdrop-blur-sm`.
- Precio formateado con `number_format((float) $producto->precio, 2)` y prefijo `S/`.
- Botón "Ver detalle" con `rounded-full` y anillo de foco `focus-visible:ring-amber-500`.
- Estado vacío cuando la colección no tiene elementos: tarjeta con borde discontinuo, icono y texto "El catálogo está en preparación".
- Paginación solo si `hasPages()`.
- La vista recibe `$productos`; no hay filtros por categoría en la UI todavía, aunque el orden y el índice están preparados para ello.

Ficha (`resources/views/productos/show.blade.php`)

- Calcula en el bloque `@php` la URL del modelo y la disponibilidad (`stock > 0`).
- Rejilla principal: una columna y, desde `lg` , dos columnas (`lg:grid-cols-2`) con separación `lg:gap-12` .
- Contenedor del visor: `[data-visor-contenedor]` con `relative aspect-square w-full overflow-hidden rounded-2xl border border-stone-200 bg-stone-100` . El fondo stone actúa como lienzo neutro porque el renderer usa alpha transparente.
- Canvas `[data-visor-3d]` absoluto, `cursor-grab active:cursor-grabbing` , `opacity-0` hasta que el modelo carga, y `pointer-events-none` cuando no hay modelo.
- Estados superpuestos:
  - `[data-visor-cargando]` : spinner animado y texto "Cargando modelo 3D…".
  - `[data-visor-pista]` : chip "Vista 3D" solo si hay modelo.
  - `[data-visor-error]` : oculto por defecto; el JS lo muestra y lo pasa a `flex` .
  - Placeholder "Modelo 3D no disponible" con icono SVG cuando no hay `modelo_3d_path` .
- Ayuda `[data-visor-ayuda]` : "Arrastra para rotar · Rueda o pinza para hacer zoom", enlazada con `aria-describedby="visor-ayuda"` .
- Columna de información: badge de categoría, `h1` , descripción con fallback, precio, badge de disponibilidad, unidades en stock con `Str::plural` .
- Formulario real "Agregar al carrito" (`POST carrito.add`) si hay stock: cantidad numérica con `min="1"` , `max="{{ min($producto->stock, 10) }}"` , `@csrf` y `producto_id` oculto. El texto inferior indica cuántas unidades se pueden agregar. Si `stock = 0` , el botón "Agotado" queda deshabilitado y no envía nada. El flujo posterior está en 04-carrito-y-pedidos.md.
- Enlace "Volver al catálogo" a `productos.index` .

Layout y navegación

- `layouts/app.blade.php` : `min-h-screen bg-stone-50 text-stone-800` , tipografía Instrument Sans autohospedada vía `@fonts` (plugin `laravel-vite-plugin/fonts` con Bunny) y `<title>` alimentado por el slot `title` de `x-app-layout` .
- `layouts/navigation.blade.php` : navbar `sticky top-0 z-50` con `bg-stone-50/80 backdrop-blur-md` , marca enlazada al catálogo, enlace "Catálogo" en escritorio y menú móvil, "Panel" solo autenticados y "Administración" solo administradores. Los invitados ven "Iniciar sesión" y "Crear cuenta". El menú móvil replica la lógica con `@auth` y `isAdmin()` .
- Ocultar "Administración" es solo cosmético; la autorización efectiva sigue en el middleware `role:admin` documentado en 02-roles-y-autenticacion.md.

API del visor 3D

Atributos `data-*`

ATRIBUTO	ELEMENTO	USO
`data-visor-3d`	`<canvas>`	Marcador que descubre el visor; el JS procesa cada coincidencia una sola vez.
`data-modelo-url`	`<canvas>`	URL absoluta del GLB. Vacía significa "sin modelo": no se inicializa WebGL.
`data-modelo-alt`	`<canvas>`	Nombre del producto; informativo, el texto accesible está en `aria-label`.
`data-visor-contenedor`	`<div>` padre	Ámbito donde el JS busca spinner, pista, error y ayuda; también se observa con `ResizeObserver`.
`data-visor-cargando`	`<div>`	Capa de carga que se oculta al terminar `onLoad`.
`data-visor-pista`	`<span>`	Chip "Vista 3D" que se oculta en error.
`data-visor-error`	`<div>`	Panel de error que el JS muestra y alterna a `flex`.
`data-visor-ayuda`	`<p>`	Texto de instrucciones; se oculta si falla la carga.

No hay opciones por atributo para color, cámara o velocidad: la configuración es fija en `visor-3d.js` para mantener consistente la experiencia. Un producto por página es el caso previsto; el código soporta varios canvas porque itera sobre todos y usa un `WeakSet` para no inicializar dos veces.

Interacción y cámara

- `OrbitControls` sobre el canvas: arrastre para rotar, rueda o pinza para zoom. `enablePan = false` evita desplazar el modelo fuera de cuadro.
- *Damping* activo (`0.08`), `rotateSpeed = 0.8` , `zoomSpeed = 0.9` .
- Auto-rotación a `0.6` salvo que el usuario prefiera menos movimiento (`prefers-reduced-motion: reduce`). La preferencia se lee una vez al inicializar.
- Cámara `PerspectiveCamera(35, 1, 0.1, 200)` . Al recibir el GLB, `Box3` obtiene tamaño y centro: el modelo se recentra en el origen y la distancia se calcula a partir de la dimensión mayor y el FOV, con un margen de `1.55` . Después se ajustan `near` , `far` , `minDistance` y `maxDistance` .
- Luces: ambiente blanca (`1.5`), direccional cálida (`0xfff2dd` , `2.6`) y direccional de relleno (`1.1`). Renderer con `ACESFilmicToneMapping` , exposición `1.15` , espacio `SRGBColorSpace` y fondo transparente.
- `ResizeObserver` sobre el contenedor más un listener de `resize` , para que el canvas siga el tamaño CSS (`aspect-square`).

- El loop usa `requestAnimationFrame`, actualiza controles y renderiza de forma continua mientras el documento siga vivo.

Accesibilidad

- El canvas tiene `role="img"` y un `aria-label` que explica el producto y las interacciones.
- Con modelo, es enfocable (`tabindex="0"`) y referencia la ayuda con `aria-describedby`; sin modelo queda `aria-hidden="true"` y no recibe puntero.
- El contenido alternativo dentro del canvas cubre navegadores sin WebGL.
- El estado vacío y el de error son HTML real, no dependen de JavaScript para existir; el placeholder se renderiza en el servidor.
- Limitación conocida: el canvas es enfocable, pero no se registran controles por teclado; la rotación requiere puntero. La información esencial (nombre, precio, stock, descripción) está siempre en la columna de texto.

Datos semilla y estado actual

`DatabaseSeeder` invoca `AdminUserSeeder` y después `ProductoSeeder`. Este último es idempotente (`updateOrCreate` por `nombre`) y crea:

PRODUCTO	PRECIO	STOCK	CATEGORÍA	`MODELO_3D_PATH`	`IMAGEN_PATH`
Espresso	2.50	40	Café	`null`	`null`
Latte	3.80	35	Café	`null`	`null`
Capuchino	3.60	30	Café	`null`	`null`

El repositorio **no incluye archivos `.glb` ni `.gltf`**. Por tanto, tras `migrate --seed` el catálogo muestra las tarjetas con el degradado y la ficha muestra el placeholder "Modelo 3D no disponible". Esto es el comportamiento esperado, no un fallo.

> *`ProductoSeeder` fuerza `modelo_3d_path` e `imagen_path` a `null` en cada ejecución. Si asignas rutas a estos tres productos y vuelves a ejecutar `php artisan db:seed`, las perderás. Para pruebas, usa otros productos o vuelve a asignar la ruta después del seed.*

Cómo cargar un modelo real

1. **Crear la carpeta pública de modelos** (aún no existe en una instalación limpia):

```
mkdir -p storage/app/public/productos
```

2. **Copiar el archivo** con un nombre sin espacios ni caracteres especiales. Se recomienda GLB binario con texturas incrustadas:

```
cp ~/Descargas/espresso.glb storage/app/public/productos/espresso.glb
```

   Recomendaciones de exportación:

   - GLB (un solo archivo) antes que glTF + `.bin` + texturas sueltas; así se evitan rutas relativas rotas.
   - Sin compresión Draco ni KTX2/meshopt: el visor solo registra `GLTFLoader`, sin `DRACOLoader` ni transcoder KTX2. Un GLB con Draco fallará al cargar.
   - Menos de 2–3 MB y unas decenas de miles de triángulos para cumplir el objetivo de carga de RNF-02.
   - Eje Y hacia arriba. La escala y el origen se normalizan automáticamente con `Box3`, pero un modelo con dimensiones absurdas puede perder precisión en `near` / `far`.

3. **Asegurar el symlink público** (si ya existe, el comando lo indica y no hace falta recrearlo):

```
php artisan storage:link
```

   El symlink requerido es `public/storage → storage/app/public`. Está en `.gitignore`, así que debe crearse en cada entorno.

4. **Guardar la ruta relativa en la base de datos**. No incluyas el prefijo `storage/` ni una barra inicial: Blade lo antepone.

```
php artisan tinker --execute 'App\Models\Producto::where("nombre", "Espresso")->update([
```

   Equivalente por SQL:

```
UPDATE productos SET modelo_3d_path = 'productos/espresso.glb' WHERE nombre = 'Espresso'
```

5. **Verificar que el archivo se sirve**:

```
ls -l public/storage
curl -I http://127.0.0.1:8000/storage/productos/espresso.glb
```

   La respuesta debe ser `200` con `Content-Type` de modelo (habitualmente `model/gltf-binary`) o `application/octet-stream`; un `404` indica symlink, ruta o nombre incorrectos.

6. **Recargar la ficha** `/productos/{id}`. Si el navegador conservó el HTML anterior, fuerza recarga. Con `npm run dev` los cambios de Blade se reflejan al instante; si modificaste JS, Vite recarga la página. En producción ejecuta `npm run build`.

La asignación manual por consola es posible, pero el CRUD con subida de archivos desde la interfaz corresponde al Módulo 4 (05-administracion.md). La ruta también puede fijarse desde la factory en pruebas (`modelo_3d_path` explícito), como hace `ProductoCatalogoTest`.

Manejo de errores

SITUACIÓN	DETECCIÓN	RESULTADO VISIBLE
Producto inactivo	`abort_unless($producto->activo, 404)`	Página 404 de Laravel.
`modelo_3d_path` nulo o vacío	Blade y comprobación de URL en JS	Placeholder "Modelo 3D no disponible"; no se crea renderer.
WebGL no disponible o el contexto falla	`try/catch` al construir `WebGLRenderer`	Canvas oculto y panel "No se pudo cargar el modelo 3D".
GLB con 404, CORS o archivo corrupto	Callback de error de `GLTFLoader`	Se ocultan canvas, spinner, pista y ayuda; se muestra el panel de error y se liberan recursos.
Excepción al calcular `Box3` o montar la escena	`try/catch` en `onLoad`	Mismo panel de error y limpieza.
Cambio de tamaño	`ResizeObserver` y evento `resize`	Se recalcula aspecto y tamaño del renderer.
Salida de la página o *back/forward cache*	`pagehide`	`dispose()` de geometrías, materiales, texturas, controles y renderer; `forceContextLoss()`.

El panel de error es un estado terminal: no hay reintento automático. El texto pide recargar o volver más tarde. Las texturas y geometrías se liberan incluso si el modelo quedó a medio construir, porque el recorrido de la escena está envuelto en `try/catch`.

Rendimiento y limitaciones

- **Three.js se importa estáticamente** desde `app.js`, así que el bundle incluye Three.js en todas las páginas, aunque no haya visor. El build actual ronda **694 kB (178 kB gzip)** para `app.js` y Vite advierte por el umbral de 500 kB. Optimización futura: `import()` dinámico de `visor-3d.js` solo cuando exista un `[data-modelo-url]` no vacío, o separar Three.js en un chunk manual. No se ha aplicado para no alterar el arranque actual.

- Un solo renderer por ficha; no hay escenas compartidas ni instancing. El caso de uso es una ficha con un producto.
- `setPixelRatio` se limita a `2`, lo que reduce el coste en pantallas de alta densidad.
- El loop de render es continuo mientras la página está viva (necesario para damping y auto-rotación). No se pausa de forma explícita al ocultar la pestaña ni al hacer scroll fuera de vista; sería una mejora con `IntersectionObserver` o `visibilitychange`.
- El modelo se sirve como archivo estático a través del symlink; no hay control de acceso ni URLs firmadas. Es adecuado para catálogo público.
- No hay compresión Draco, KTX2 ni meshopt, ni generación de LOD o *fallback* geométrico. La calidad del GLB es responsabilidad del archivo que se suba.
- No hay precarga del modelo ni *skeleton* con progreso porcentual: el spinner es indeterminado.
- La auto-rotación solo respeta `prefers-reduced-motion` en el momento de inicializar; un cambio en caliente de la preferencia no se detecta.
- Si JavaScript no se ejecuta en una ficha que sí tiene modelo, el spinner HTML permanece visible y el canvas nunca aparece; el contenido textual de la ficha sigue siendo legible.
- El listado no filtra por categoría en la interfaz ni carga imágenes reales de los productos semilla; la estructura y el índice están preparados.
- No hay medición automatizada ni presupuesto de rendimiento en CI: el objetivo de menos de 3 segundos de RNF-02 no está instrumentado. Los factores determinantes son el peso del bundle de Three.js y el tamaño del GLB.
- El botón de carrito está deshabilitado si `stock = 0`; con stock, envía el formulario real a `carrito.add` (Módulo 3). El límite de 10 unidades del input es solo de interfaz: el servidor valida contra el stock disponible.

Pruebas

```
# Solo el módulo de catálogo
php artisan test --filter=ProductoCatalogoTest

# Suite completa
php artisan test --compact
```

Cobertura de `tests/Feature/ProductoCatalogoTest.php`:

PRUEBA	ESCENARIO	RESULTADO ESPERADO
`test_el_catalogo_solo_muestra_productos_activos`	Un producto activo y otro inactivo.	El activo aparece; el inactivo no.

`test_el_detalle_muestra_un_producto_activo`	Ficha de un producto activo.	`200`, nombre visible y botón "Agregar al carrito".
`test_el_detalle_devuelve_404_para_un_producto_inactivo`	Ficha de un producto inactivo.	`404`.
`test_el_detalle_incluye_el_visor_3d_con_la_url_del_modelo`	Producto con `modelo_3d_path`.	`data-visor-3d`, `data-modelo-url` con `asset('storage/...')` y sin placeholder.
`test_el_detalle_muestra_el_estado_vacio_cuando_no_hay_modelo`	Producto sin modelo.	`data-modelo-url=""` y texto "Modelo 3D no disponible".

Estado global verificado: **48 pruebas aprobadas, 172 aserciones**, incluidas las 8 pruebas de carrito y las 7 de pedidos del Módulo 3. Las pruebas de este documento son HTTP y no ejecutan WebGL: validan el HTML y los atributos que consume el visor, no el render en navegador.

Assets: desarrollo y producción

```
# Desarrollo con recarga en caliente (Vite dev server)
npm run dev

# Build de producción; Laravel consume public/build/manifest.json
npm run build
```

`public/build` está en `.gitignore`, por lo que el build debe generarse en cada entorno. Si la ficha no refleja un cambio de Blade o aparece el error de Vite "Unable to locate file in Vite manifest", regenera el build o mantén `npm run dev` activo.

Verificación manual sugerida

1. `npm install && npm run dev` y, en otra terminal, `php artisan serve`.
2. Abrir `http://127.0.0.1:8000/` y comprobar el grid en 375 px, 768 px y 1280 px.
3. Entrar a un producto semilla: debe verse el placeholder, sin spinner ni errores en consola.
4. Cargar un GLB siguiendo la guía anterior y recargar: spinner, aparición del modelo y auto-rotación.
5. Arrastrar, usar rueda y pinza; comprobar que el paneo está desactivado y que el zoom tiene límites.
6. Renombrar temporalmente el GLB para provocar un 404: debe aparecer "No se pudo cargar el modelo 3D" y desaparecer el canvas.
7. En DevTools, emular `prefers-reduced-motion: reduce` y recargar: el modelo no debe auto-rotar.
8. Marcar un producto como inactivo (`activo => false`) y abrir su URL: debe responder 404.

Referencias

- Esquema de `productos` y convención de rutas de archivos: 01-base-de-datos.md.
- Autenticación, roles y protección de `/admin`: 02-roles-y-autenticacion.md.
- Carrito y checkout que consumen el catálogo: 04-carrito-y-pedidos.md.
- Puesta en marcha y estado general: README.md.

04 — Carrito y pedidos

Este documento describe el módulo de compra: el carrito de sesión y la confirmación transaccional de pedidos. Cubre **RF-03** (carrito de compras) y **RF-04** (registro del pedido). El flujo es público: un invitado puede agregar productos y confirmar sin crear cuenta; un usuario autenticado obtiene además la asociación del pedido a su cuenta.

El módulo se apoya en el catálogo descrito en 03-catalogo-y-visor-3d.md y en las tablas `pedidos` y `detalle_pedido` documentadas en 01-base-de-datos.md. No se modificó el esquema de autenticación de 02-roles-y-autenticacion.md.

Alcance y requerimientos

REQUERIMIENTO	IMPLEMENTACIÓN ACTUAL
RF-03 — carrito de compras	Carrito almacenado en la sesión con estructura `[producto_id => cantidad]`; agregar, restar, quitar y vaciar; precios y subtotales recalculados desde la base de datos en cada vista.
RF-04 — registro del pedido	Checkout con método de pago y notas; creación de `pedidos` y `detalle_pedido` dentro de una transacción; descuento de stock; confirmación con URL firmada.

Fuera de alcance de este módulo:

- Pasarela de pago real. `metodo_pago` registra una preferencia; el cobro se resuelve en tienda.
- Historial de pedidos para el cliente y gestión administrativa de pedidos (Módulo 4).
- Reserva de stock: el stock solo se descuenta al confirmar el pedido.
- Persistencia del carrito entre dispositivos o tras cerrar sesión.

Componentes del módulo

COMPONENTE	ARCHIVO	RESPONSABILIDAD
Carrito	`app/Carrito.php`	Lee, normaliza, resuelve y modifica el carrito de sesión.
Controlador del carrito	`app/Http/Controllers/CarritoController.php`	`index`, `add`, `remove`, `clear`; valida producto, estado y stock.
Controlador de pedidos	`app/Http/Controllers/PedidoController.php`	`store` (checkout transaccional) y `confirmado` (vista firmada).
Enum de pago	`app/Enums/MetodoPago.php`	Valores `efectivo`, `tarjeta`, `transferencia`.
Modelos	`app/Models/Pedido.php`, `app/Models/DetallePedido.php`	Relaciones, `#[Fillable]` y casts (`estado`, `metodo_pago`, importes).
Rutas	`routes/web.php`	Rutas públicas de carrito y pedidos.

Vista del carrito	`resources/views/carrito/index.blade.php`	Líneas, resumen, método de pago, notas y confirmación.
Vista de confirmación	`resources/views/pedidos/confirmado.blade.php`	Número, total, método y mensaje para invitados.
Mensajes flash	`resources/views/components/flash-banner.blade.php`	Banner global de éxito/error incluido en el layout.
Navegación	`resources/views/layouts/navigation.blade.php`	Icono de carrito con contador en escritorio y móvil.
Migración	`database/migrations/2026_11_08_163735_add_metodo_pago_and_allow_guest_user_to_pedidos_table.php`	Hace `pedidos.user_id` nullable y agrega `metodo_pago`.
Pruebas	`tests/Feature/CarritoTest.php`, `tests/Feature/PedidoTest.php`	Cobertura HTTP del carrito y del checkout.

Carrito en la sesión

Estructura

La sesión guarda únicamente identificadores y cantidades bajo la clave `carrito`:

```
// Contenido de session('carrito')
[
    12 => 2, // producto_id => cantidad
    7  => 1,
]
```

`Carrito::cantidades()` normaliza el arreglo: descarta claves o cantidades no positivas y convierte los valores a entero. Si la sesión no contiene un arreglo válido, devuelve `[]`. No se guardan nombres, precios, subtotales ni stock.

`Carrito::lineas()` resuelve las líneas contra la base de datos:

1. Consulta los productos con `whereIn('id', ...)->where('activo', true)`.
2. Purga de la sesión los productos inactivos o eliminados mediante `purgarNoDisponibles()`.
3. Devuelve una colección con `producto`, `cantidad`, `precio_unitario` y `subtotal`, calculados con el precio actual de `productos.precio`.

MÉTODO	ENTRADA	EFECTO
`cantidades()`	—	Devuelve `array<int, int>` normalizado.
`lineas()`	—	Devuelve las líneas con datos y precios actuales; purga productos no disponibles.
`agregar($productoId, $cantidad)`	IDs y cantidad positivos	Suma unidades a la línea existente o la crea.
`quitar($productoId, $cantidad = null)`	Sin cantidad elimina la línea; con cantidad resta unidades y elimina si el resto es cero o negativo.	Modifica la sesión.
`vaciar()`	—	Ejecuta `session()->forget('carrito')`.

Por qué no existe una tabla de carrito

El carrito es un estado efímero de la compra, no un registro de negocio. Mantenerlo en sesión aporta:

- **Sin tabla de negocio**: cada clic modifica un arreglo en la sesión; no se insertan ni actualizan filas de carrito. La sesión se persiste según `SESSION_DRIVER` (en este proyecto, `database`).
- **Sin duplicación de la verdad**: el precio y el stock viven solo en `productos`; el carrito no puede fijar una copia obsoleta de los importes.
- **Sin limpieza de carritos abandonados**: no hay filas huérfanas que purgar.
- **Compatibilidad con invitados**: la sesión ya identifica al visitante sin exigir una cuenta.

El proyecto usa `SESSION_DRIVER=database`, por lo que la sesión se persiste en la tabla `sessions`; aun así, no existe una tabla de carrito con semántica de negocio.

Contrapartidas asumidas:

- El carrito no se comparte entre dispositivos ni navegadores.
- Se pierde al cerrar sesión: el logout de Breeze ejecuta `session()->invalidate()`. En cambio, el login y el registro solo regeneran el ID de sesión, por lo que el carrito de un invitado sobrevive si decide crear una cuenta antes de confirmar.
- No hay analítica de carritos abandonados.
- Si en el futuro se requiere persistencia por usuario, habría que añadir tablas `carritos` / `carrito_items` y migrar la sesión.

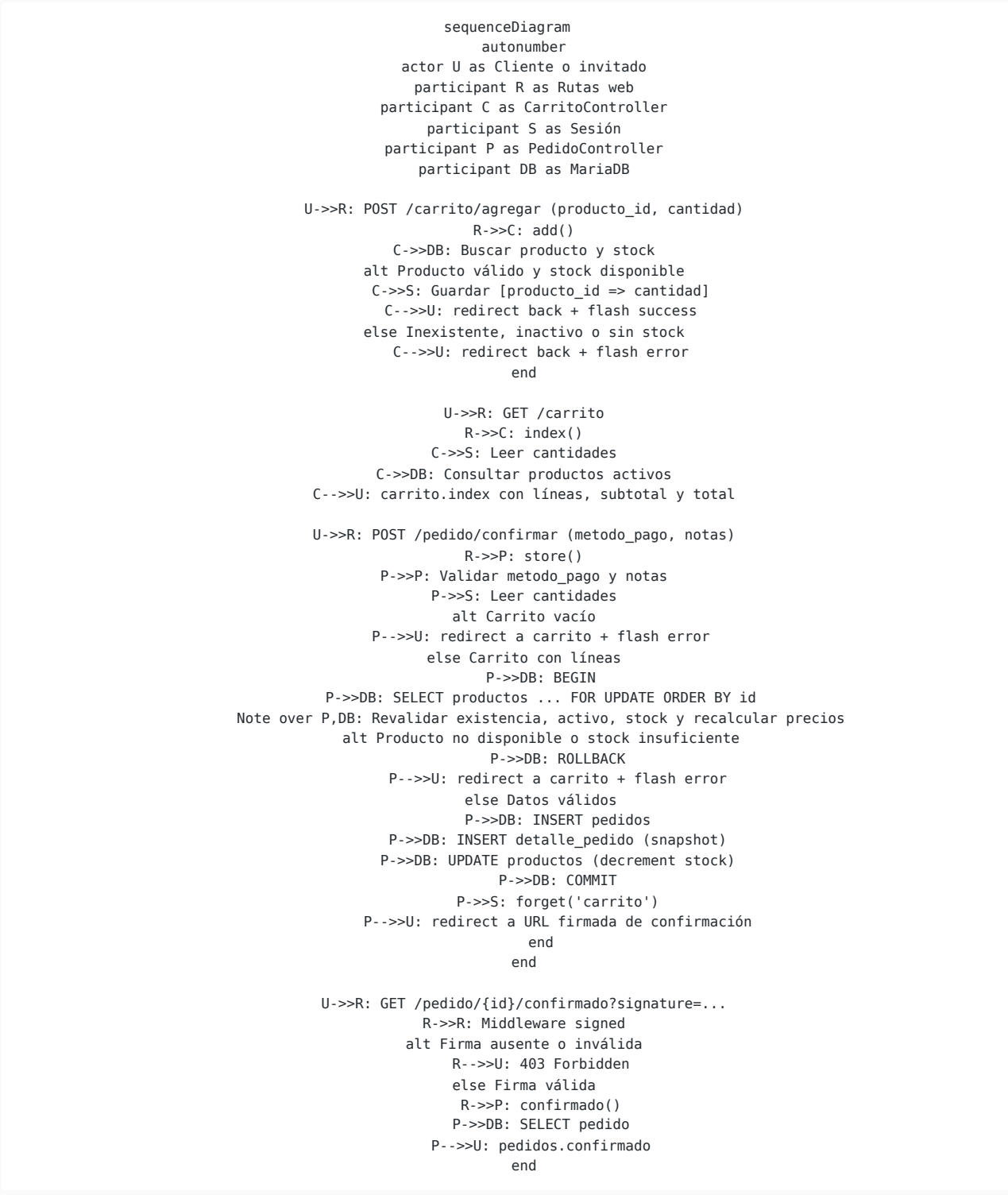
Rutas

Todas las rutas son públicas (`web`); no exigen autenticación. La única protección adicional es la firma de la confirmación.

MÉTODO	URI	NOMBRE	MIDDLEWARE	ACCIÓN
GET	`/carrito`	`carrito.index`	`web`	`CarritoController@index`
POST	`/carrito/agregar`	`carrito.add`	`web`	`CarritoController@add`
POST	`/carrito/quitar`	`carrito.remove`	`web`	`CarritoController@remove`
POST	`/carrito/vaciar`	`carrito.clear`	`web`	`CarritoController@clear`
POST	`/pedido/confirmar`	`pedidos.store`	`web`	`PedidoController@store`
GET	`/pedido/{pedido}/confirmado`	`pedidos.confirmado`	`web`, `signed`	`PedidoController@confirmado`

Todas las operaciones de escritura usan `POST` y los formularios incluyen `@csrf`. No hay rutas `PUT` / `DELETE`: agregar, restar, quitar y vaciar se expresan como envíos de formulario.

Flujo de checkout



Resumen del camino feliz:

1. El cliente agrega unidades desde la ficha o desde el propio carrito.
2. `GET /carrito` reconstruye las líneas con precios actuales y muestra el resumen.
3. `POST /pedido/confirmar` valida pago y notas, y comprueba que el carrito no esté vacío.
4. Dentro de una transacción se bloquean los productos, se revalida stock y se calcula el total.

5. Se insertan cabecera y detalles, se descuenta stock y se confirma la transacción.
6. Se vacía la sesión y se redirige a una URL firmada con el número de pedido.

Stock, precios y totales

Frontera de confianza

El navegador solo envía `producto_id` y `cantidad` (o `metodo_pago` y `notas`). **Nunca** se acepta un precio, subtotal ni total desde el cliente. El servidor resuelve el producto y calcula los importes con `productos.precio`.

Validaciones por etapa

ETAPA	ARCHIVO	COMPROBACIONES
Agregar	`CarritoController@add`	`producto_id` entero y existente; `cantidad` entero ≥ 1; producto activo; `stock > 0`; `cantidad ≤ stock - unidades ya en carrito`.
Ver carrito	`Carrito::lineas()`	Solo productos activos; purga inactivos/eliminados de la sesión; recalcula precio y subtotal actuales. No recorta cantidades al stock actual.
Confirmar	`PedidoController@store`	Carrito no vacío; producto existente; producto activo; `stock ≥ cantidad`; método de pago permitido; notas ≤ 500 caracteres.
Persistencia	Transacción	`lockForUpdate()` sobre los productos ordenados por `id`; inserción de cabecera/detalles; `decrement('stock', $cantidad)`.

Detalles relevantes:

- **Stock al agregar**: se descuenta lo que ya está en el carrito antes de comparar. Si un producto tiene stock 3 y el carrito ya tiene 2, solo se puede agregar 1 unidad más.
- **Stock al confirmar**: aunque el carrito se haya construido con stock válido, otro pedido pudo consumirlo. Por eso se vuelve a validar dentro de la transacción.
- **Bloqueo ordenado**: `orderBy('id')->lockForUpdate()` bloquea las filas en un orden consistente para reducir el riesgo de interbloqueos entre checkouts concurrentes.
- **Productos retirados**: si un producto se desactiva o elimina después de agregarlo, `lineas()` lo purga al abrir el carrito. Si la sesión aún lo contiene al confirmar, el checkout lanza un error de validación y no crea nada.
- **Cantidades no recortadas**: el carrito puede contener una cantidad superior al stock actual si el stock bajó después de agregar. La vista lo advierte con “Has alcanzado el stock disponible” y el checkout lo rechaza; no se modifica la sesión silenciosamente.
- **Límite de 10 unidades**: la ficha aplica `max="{{ min($producto->stock, 10) }}"` como límite de interfaz. El servidor no impone ese tope: valida únicamente contra el stock disponible. Un envío manipulado puede agregar más de 10 unidades si hay stock.
- **Redondeo**: cada subtotal se calcula como `round($precioUnitario * $cantidad, 2)`; el total es la suma de subtotales redondeada a dos decimales. La persistencia usa `decimal(10,2)`.

Transacción y atomicidad

`PedidoController@store` envuelve la creación completa en `DB::transaction()`. Si cualquier producto falla la revalidación, se lanza `ValidationException`; la transacción se revierte y no quedan pedidos, detalles ni descuentos de stock parciales. La sesión conserva el carrito para que el cliente pueda corregirlo.

La cabecera se crea con `EstadoPedido::Pendiente`, el `user_id` de `auth()->id()` (o `NULL` para invitados), el `metodo_pago` validado, el total calculado y las notas opcionales. Los detalles se insertan con `createMany()` y el stock se descuenta con `decrement()` dentro de la misma transacción.

Las únicas excepciones capturadas son las de validación. Un fallo de base de datos distinto (por ejemplo, una conexión caída) se propaga como error 500, pero la transacción garantiza que no deja escrituras a medias.

Invitado frente a autenticado

ASPECTO	INVITADO	AUTENTICADO
Agregar y ver carrito	Sí, con la sesión anónima.	Sí, misma clave de sesión.
Confirmar pedido	Sí.	Sí.
`pedidos.user_id`	`NULL`.	`auth()->id()`.
Mensaje de confirmación	“Pedido registrado como invitado.”	“Pedido registrado correctamente.”
Aviso en la vista	“Guarda tu número de pedido para consultarlo en tienda.”	No se muestra.
Botón “Ver mi panel”	No.	Sí, enlace a `dashboard`.
Acceso a la confirmación	URL firmada.	URL firmada; no se comprueba la propiedad del pedido.
Historial de pedidos	No existe todavía.	No existe todavía.

La decisión de permitir checkout sin login evita fricción en una cafetería y mantiene el flujo de Breeze como opcional. Para soportarlo, `pedidos.user_id` pasó a ser nullable conservando la FK `RESTRICT`: un pedido de invitado no tiene usuario, pero sigue siendo imposible borrar una cuenta con pedidos asociados.

Consecuencias:

- `User::pedidos()` solo devuelve pedidos con `user_id`; los pedidos de invitados no aparecen en ninguna relación de usuario.
- No se puede identificar al invitado más allá de la URL firmada de confirmación.
- La vista de confirmación es deliberadamente genérica: no muestra líneas, notas ni datos personales.

Método de pago y notas

El método de pago se modela con `App\Enums\MetodoPago`:

CASO	VALOR PERSISTIDO	ETIQUETA EN LA VISTA
`MetodoPago::Efectivo`	`efectivo`	Efectivo
`MetodoPago::Tarjeta`	`tarjeta`	Tarjeta
`MetodoPago::Transferencia`	`transferencia`	Transferencia

- `PedidoController@store` valida `metodo_pago` con `required|in:efectivo,tarjeta,transferencia`.
- La UI preselecciona `efectivo` mediante `old('metodo_pago', 'efectivo')` y recorre `MetodoPago::cases()`.
- `pedidos.metodo_pago` es `varchar(255)` con default `'efectivo'`; el modelo lo castea a `MetodoPago`, por lo que `$pedido->metodo_pago` es un enum y la vista usa `->value`.
- No hay cobro ni integración con pasarela: es una preferencia para preparar el pedido y cobrar en tienda.
- `notas` es opcional (`nullable|string|max:500`). Blade escapa el contenido al mostrarlo; la vista de confirmación no las muestra.
- La columna `notas` es `TEXT`; el límite de 500 es una regla de aplicación, no del esquema.

Snapshot de precios

`detalle_pedido` guarda una copia del precio y del subtotal al momento de la compra:

COLUMNA	ORIGEN
`producto_id`	Producto del carrito.
`cantidad`	Cantidad de la sesión.

`precio_unitario`	`productos.precio` en el momento del checkout.
`subtotal`	`cantidad × precio_unitario`, redondeado a dos decimales.

El total del pedido se consolida en `pedidos.total`. Así, un cambio posterior de precio no altera el importe histórico. Ejemplo: 3 unidades de un producto de S/ 12.50 generan un detalle con `precio_unitario = 12.50`, `subtotal = 37.50` y un pedido con `total = 37.50`.

La sesión nunca almacena estos importes; el snapshot se crea únicamente al confirmar, con los valores leídos dentro de la transacción.

Confirmación firmada

Tras el checkout, el controlador redirige a:

```
URL::signedRoute('pedidos.confirmado', ['pedido' => $pedido->getKey()])
```

La ruta `GET /pedido/{pedido}/confirmado` lleva el middleware `signed`. Comportamiento:

- Firma ausente o inválida: `403 Forbidden`, incluso si el `id` no existe (el middleware se ejecuta antes del controlador).
- Firma válida: `PedidoController@confirmado` busca el pedido con `findOrFail`; un `id` inexistente con firma válida produce `404`.
- La URL no caduca: se usa `signedRoute`, no `temporarySignedRoute`.
- La firma demuestra que la URL fue generada por la aplicación, **no** que el visitante sea el dueño del pedido. Quien tenga el enlace puede ver número, total y método de pago. Por eso la vista no incluye detalles, notas ni datos del cliente.
- No se exige login para ver la confirmación; es coherente con el checkout de invitado.

La vista muestra:

- Número de pedido (`#{{ $pedido->id }}`).
- Total formateado (`S/ {{ number_format((float) $pedido->total, 2) }}`).
- Método de pago con inicial mayúscula (`ucfirst($pedido->metodo_pago->value)`).
- Aviso de pedido como invitado si `user_id === null`.
- Botón “Volver al catálogo” y, solo para usuarios autenticados, “Ver mi panel”.

Manejo de errores

SITUACIÓN	DETECCIÓN	RESULTADO
Producto inexistente al agregar	`exists:productos,id`	Error de validación en `producto_id`; el banner muestra el primer error.
Producto inactivo al agregar	Comprobación de `activo`	`redirect back` con flash `error`; no se modifica el carrito.
Stock insuficiente al agregar	`stock - enCarrito`	`redirect back` con mensaje “No hay stock suficiente…”; indica el máximo disponible.
Producto agotado al agregar	`stock <= 0` o disponible `<= 0`	Flash de error; el carrito queda intacto.
Cantidad inválida	Validación `integer` + `min:1`	Redirección con errores de validación; el layout muestra el primer error.
Carrito vacío al confirmar	`cantidades() === []`	Redirect a `carrito.index` con flash de error.
Producto eliminado al confirmar	`$productos->get($id) === null`	`ValidationException` → rollback → redirect al carrito con error.
Producto inactivo al confirmar	`! $producto->activo`	`ValidationException` → rollback → redirect al carrito con error.
Stock insuficiente al confirmar	`$producto->stock < $cantidad`	`ValidationException` → rollback → redirect al carrito con error; el carrito se conserva.

Método de pago inválido	`in:efectivo,tarjeta,transferencia`	Error de validación; no se crea pedido.
Notas demasiado largas	`max:500`	Error de validación; no se crea pedido.
URL de confirmación sin firma	Middleware `signed`	`403 Forbidden`.
Pedido inexistente con firma válida	`findOrFail`	`404 Not Found`.
Fallo de base de datos	Excepción no capturada	`500`; la transacción revierte.

Los errores de checkout se devuelven con `with('error', ...)` y `withErrors(...)`, de modo que el banner global y el formulario pueden mostrar el mensaje. El carrito no se vacía ante un fallo.

Interfaz

Carrito (`resources/views/carrito/index.blade.php`)

- Estética stone/amber coherente con el catálogo: tarjetas `rounded-2xl`, bordes `stone-200`, acentos `amber-600`.
- Cabecera con enlace “Volver al catálogo”, título “Tu carrito”, contador de unidades y botón “Vaciar carrito” solo si hay líneas.
- Estado vacío con borde discontinuo, icono de taza, mensaje y botón “Explorar el catálogo”.
- Rejilla de dos columnas desde `lg`: líneas a la izquierda y resumen sticky (`lg:sticky lg:top-24`) a la derecha.
- Cada línea muestra imagen o degradado, nombre enlazado a la ficha, precio unitario, subtotal y aviso si se alcanzó el stock.
- Control de cantidad con dos formularios `POST`: uno resta 1 (`carrito.remove` con `cantidad=1`) y otro suma 1 (`carrito.add`). El botón “+” se deshabilita cuando `cantidad >= stock`.
- Botón “Quitar” que elimina la línea completa (`carrito.remove` sin `cantidad`).
- Resumen: subtotal, “Recojo en tienda” como envío y total; ambos importes se recalculan en el servidor.
- Formulario de checkout con radios de método de pago en tarjetas (`has-[:checked]`), notas con `maxlength="500"` y botón “Confirmar pedido”.
- Enlace “Seguir comprando” al catálogo.

Contador del navbar (`layouts/navigation.blade.php`)

- El icono del carrito aparece para invitados y autenticados, en escritorio y en el menú móvil.
- El badge ámbar muestra la suma de cantidades de la sesión (`array_sum`), con tope visual `99+`.
- El atributo `data-carrito-contador` expone el valor para las pruebas HTTP.
- El contador lee la sesión cruda, no `Carrito::lineas()`: un producto inactivo o eliminado puede seguir sumando hasta que se abra `/carrito` y se purgue la sesión.

Ficha de producto (`productos/show.blade.php`)

- Si `stock > 0`, renderiza un formulario real a `carrito.add` con `@csrf`, `producto_id` oculto y cantidad numérica.
- `max` del input es `min(stock, 10)` y el texto inferior indica “Puedes agregar hasta N unidades por pedido”.
- Si `stock = 0`, el botón “Agotado” queda deshabilitado y no se envía nada.
- Los errores de validación de `cantidad` se muestran bajo el formulario.

Confirmación (`pedidos/confirmado.blade.php`)

- Tarjeta centrada con icono de check, título “¡Gracias por tu pedido!” y tres bloques: pedido, total y método de pago.
- Aviso ámbar para invitados.
- Acciones: “Volver al catálogo” siempre; “Ver mi panel” solo con `@auth`.

Mensajes flash (`components/flash-banner.blade.php`)

- Incluido globalmente en `layouts/app.blade.php` , después de la navegación.
- Muestra `session('success')` en un banner ámbar y `session('error')` en un banner neutro con icono rojo.
- Si no hay `error` explícito pero existen errores de validación, muestra el primero (`$errors->first()`).
- Usa `role="status"` / `aria-live` para el éxito y `role="alert"` para el error.

Pruebas

```
# Carrito
php artisan test --filter=CarritoTest

# Pedidos y confirmación
php artisan test --filter=PedidoTest

# Suite completa
php artisan test --compact
```

Cobertura de `tests/Feature/CarritoTest.php` (8 pruebas):

PRUEBA	ESCENARIO	RESULTADO ESPERADO
`test_agregar_un_producto_actualiza_la_sesion_y_el_contador`	Agregar 2 unidades desde la ficha.	Sesión `[id => 2]` , flash de éxito y badge `data-carrito-contador="2"` .
`test_no_permite_agregar_mas_unidades_de_las_disponibles`	Stock 3, carrito con 2, intento de 2 más.	Flash de error y carrito intacto en 2.
`test_rechaza_un_producto_inactivo`	Producto con `activo = false` .	Flash de error; no se agrega.
`test_la_cantidad_debe_ser_al_menos_uno`	`cantidad = 0` .	Error de validación en `cantidad` .
`test_quitar_una_unidad_y_eliminar_la_linea_completa`	Restar 1 y luego quitar sin cantidad.	Pasa de 3 a 2 y luego la sesión queda vacía.
`test_vaciar_el_carrito`	`POST /carrito/vaciar` .	Sesión `null` y flash de éxito.
`test_el_carrito_ignora_productos_inactivos_o_eliminados`	Sesión con un activo, un inactivo y un ID inexistente.	Solo se muestra el activo y la sesión se purga.
`test_el_contador_del_navbar_suma_las_cantidades`	Dos productos con 2 y 3 unidades.	Badge `data-carrito-contador="5"` .

Cobertura de `tests/Feature/PedidoTest.php` (7 pruebas):

PRUEBA	ESCENARIO	RESULTADO ESPERADO
`test_confirmar_crea_un_pedido_pendiente_con_detalles_y_descuenta_stock`	Usuario autenticado, 3 unidades, pago con tarjeta y notas.	Pedido `pendiente` , total `37.50` , detalle con snapshot, stock descontado y sesión vacía; redirección firmada.
`test_un_invitado_puede_confirmar_y_el_pedido_queda_sin_usuario`	Checkout sin autenticación, pago efectivo.	`user_id = null` , total `8.00` , stock descontado y aviso de invitado en la confirmación.
`test_stock_insuficiente_no_crea_pedido_ni_descuenta_stock`	Carrito con 5 unidades y stock 2.	Error, cero pedidos/detalles, stock intacto y carrito conservado.
`test_un_producto_inactivo_en_el_carrito_bloquea_la_confirmacion`	Producto inactivo en la sesión.	Error, sin pedido y sin descuento.
`test_metodo_pago_invalido_falla_la_validacion`	`metodo_pago = bitcoin` .	Error de validación, sin pedido ni descuento.

`test_no_se_puede_confirmar_con_el_carrito_vacio`	Checkout sin líneas.	Redirect al carrito con flash de error; cero pedidos.
`test_la_confirmacion_requiere_una_url_firmada`	Confirmación sin firma y con firma para un ID inexistente.	`403` en ambos casos; con firma válida muestra número, total y método.

Estado global verificado de la suite: **48 pruebas aprobadas y 172 aserciones**. Las pruebas son HTTP y no ejecutan JavaScript: validan sesión, base de datos, redirecciones, firmas y HTML; el comportamiento visual del carrito y del badge debe comprobarse en navegador.

Verificación manual

1. Preparar el entorno:

   ```
   php artisan migrate --seed
   npm install
   npm run dev
   php artisan serve
   ```

   Si no se usa Vite en modo desarrollo, ejecutar `npm run build`.

2. Abrir `http://127.0.0.1:8000/`, entrar a un producto con stock y agregar 2 unidades. Debe aparecer el banner de éxito y el badge del navbar en `2`.
3. Abrir `/carrito`: comprobar líneas, subtotal, total, botones `+` / `−`, “Quitar” y “Vaciar carrito”. El botón `+` debe deshabilitarse al alcanzar el stock.
4. Como invitado, elegir método de pago, escribir una nota y confirmar. Debe redirigir a `/pedido/{id}/confirmado?signature=...`, mostrar número, total, método y el aviso “Pedido registrado como invitado”.
5. Verificar en base de datos que el pedido tiene `user_id NULL`, estado `pendiente`, total correcto y detalles con `precio_unitario` / `subtotal`. Comprobar que el stock bajó:

   ```
   php artisan tinker --execute 'App\Models\Pedido::latest("id")->first()->load("detallesPedido");'
   ```

6. Repetir el checkout con la cuenta `cliente@cafeteria.test` / `password`. El pedido debe tener `user_id`, y la confirmación debe mostrar “Pedido registrado correctamente” y el botón “Ver mi panel”.
7. Provocar stock insuficiente: agregar 2 unidades, reducir el stock a 1 con tinker y confirmar. Debe volver al carrito con error, sin pedido y sin descuento; el carrito debe conservar las 2 unidades.
8. Provocar producto no disponible: desactivar un producto que esté en el carrito y abrir `/carrito`. La línea debe desaparecer y la sesión purgarse.
9. Abrir `/pedido/1/confirmado` sin `signature`: debe responder `403`. Repetir con la URL firmada generada por la aplicación: debe responder `200`.
10. Comprobar que el carrito sobrevive al login (la sesión se regenera sin perder datos) y que se pierde al cerrar sesión (Breeze invalida la sesión).

Limitaciones y siguientes pasos

- No hay pasarela de pago ni estados de cobro; `metodo_pago` es informativo.
- No hay historial de pedidos para el cliente ni detalle del pedido en la confirmación.
- No hay reserva de stock entre el carrito y la confirmación: el stock se descuenta al confirmar; si dos checkouts compiten, el bloqueo serializa las operaciones y el que ya no encuentra stock suficiente falla.

- La URL de confirmación es permanente y no está vinculada al usuario; quien la posea puede ver el resumen.
- El badge del navbar usa la sesión cruda y puede incluir productos que aún no se purgaron.
- El límite de 10 unidades de la ficha es solo de interfaz; el servidor valida contra el stock.
- La gestión administrativa de pedidos (cambio de estado, listado, cancelación) corresponde al Módulo 4.

La gestión de estados de los pedidos desde la interfaz administrativa (`/admin/pedidos` y cambio a `pendiente` / `en_preparacion` / `entregado`) está documentada en 05-administracion.md.

Referencias

- Esquema de `pedidos` y `detalle_pedido`, y decisión de `user_id` nullable: 01-base-de-datos.md.
- Autenticación, sesión y logout: 02-roles-y-autenticacion.md.
- Catálogo, ficha y formulario de origen: 03-catalogo-y-visor-3d.md.
- Puesta en marcha y estado general: README.md.

05 — Administración (Módulo 4)

Este documento describe el panel de administración del sistema de ventas. Cubre **RF-05** (CRUD de productos con subida de modelos 3D), **RF-06** (gestión de pedidos y cambio de estado) y **RNF-03** (acceso al panel restringido al rol `admin`).

El panel está accesible bajo el prefijo `/admin`, protegido con los middlewares `auth` y `role:admin` configurados en el Módulo 1. No se modificaron migraciones ni modelos: todos los campos necesarios (`imagen_path`, `modelo_3d_path`, `metodo_pago`, `user_id` nullable) ya existían del Módulo 1 al 3.

Alcance y requerimientos

REQUERIMIENTO	IMPLEMENTACIÓN ACTUAL
RF-05 — CRUD de productos	`Admin\AdminProductoController` con `index`, `create`, `store`, `edit`, `update`, `destroy`; subida de imagen y modelo `.glb/.gltf` a `storage/app/public/productos/` y persistencia de la ruta relativa en `imagen_path` y `modelo_3d_path`.
RF-06 — Gestión de pedidos	`Admin\AdminPedidoController` con `index` y `updateEstado`; transición de estados entre `pendiente`, `en_preparacion` y `entregado`.
RNF-03 — acceso restringido	Grupo `/admin` con `['auth','role:admin']`; invitado redirige a login, cliente recibe 403, administrador accede.

Seguridad y rutas

Todas las rutas de administración viven dentro de:

```
Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () { ... });
```

MÉTODO	URI	NOMBRE	CONTROLADOR	DESCRIPCIÓN
GET	`/admin`	`admin.dashboard`	`AdminDashboardController`	Panel principal con métricas
GET	`/admin/productos`	`admin.productos.index`	`Admin\AdminProductoController`	Listado paginado de productos
GET	`/admin/productos/create`	`admin.productos.create`	`Admin\AdminProductoController`	Formulario de nuevo producto
POST	`/admin/productos`	`admin.productos.store`	`Admin\AdminProductoController`	Crear producto con archivos
GET	`/admin/productos/{producto}/edit`	`admin.productos.edit`	`Admin\AdminProductoController`	Formulario de edición
PUT/PATCH	`/admin/productos/{producto}`	`admin.productos.update`	`Admin\AdminProductoController`	Actualizar producto y archivos

DELETE	`/admin/productos/{producto}`	`admin.productos.destroy`	`Admin\AdminProductoController`	Eliminar producto y sus archivos
GET	`/admin/pedidos`	`admin.pedidos.index`	`Admin\AdminPedidoController`	Listado de pedidos con filtro de estado
PATCH	`/admin/pedidos/{pedido}/estado`	`admin.pedidos.updateEstado`	`Admin\AdminPedidoController`	Cambiar estado de un pedido

Las vistas públicas (`/` , `/productos/{producto}` , `/carrito` , `/pedido/confirmar`) no usan `auth` ni `role:admin` y no se alteraron.

Comportamiento por sesión

- **Invitado**: al entrar a cualquier ruta `/admin` , recibe redirección (302) a `/login` .
- **Cliente autenticado**: recibe `403` al acceder a `/admin` o a cualquiera de sus subrutas.
- **Administrador autenticado**: recibe `200` y opera el panel con normalidad.

La autorización efectiva está en el middleware `role:admin` (definido en `app/Http/Middleware/EnsureUserHasRole.php` y registrado como alias `role` en `bootstrap/app.php`). El enlace "Panel de Admin" del navbar público solo se renderiza con `@if (auth()->user()?->isAdmin())` , pero es solo un acceso rápido; la autorización real está en el middleware.

CRUD de productos (RF-05)

Validación

Se usan dos Form Requests: `App\Http\Requests\StoreProductoRequest` y `App\Http\Requests\UpdateProductoRequest` .

CAMPO	REGLAS
`nombre`	`required
`descripcion`	`nullable
`precio`	`required
`stock`	`required
`categoria`	`nullable
`activo`	`sometimes
`imagen`	`nullable
`modelo_3d`	`nullable

Decisión de validación: el modelo 3D valida con `extensions:glb,gltf` porque los navegadores suelen enviar `.glb` como `application/octet-stream` ; validar solo por MIME rechazaría archivos válidos. La imagen sí valida contenido real con `image|mimes` . Los límites son 4 MB para imagen y 20 MB para modelo 3D.

> *Nota de entorno: el `php.ini` de desarrollo tiene `upload_max_filesize=24M` y `post_max_size=32M` . En producción deben ajustarse para permitir archivos de hasta 20 MB.*

Almacenamiento

Los archivos se guardan con el disco `public` de Laravel:

```php
Storage::disk('public')->putFile('productos', $archivo);
```

- Nombre único generado automáticamente por Laravel (`putFile`).
- En BD se persiste la **ruta relativa**, por ejemplo `productos/a1b2c3.glb`, en `modelo_3d_path` (y `imagen_path` para la imagen).
- El archivo físico queda en `storage/app/public/productos/` y se sirve mediante el symlink existente `public/storage → storage/app/public`, que permite al visor público cargar `asset('storage/productos/a1b2c3.glb')`.

Flujo de subida (diagrama Mermaid)

```
flowchart TD
    A[Admin abre /admin/productos/create] --> B[Formulario multipart con imagen y modelo_3d]
    B --> C[POST /admin/productos + StoreProductoRequest]
    C --> D{"¿Validación OK?"}
    D -->|No| E[Redirige con errores inline, conserva entrada]
    D -->|Sí| F[Guardar imagen con Storage::disk('public')->putFile]
    F --> G[Guardar modelo 3D con putFile]
    G --> H[Persistir rutas relativas en imagen_path y modelo_3d_path]
    H --> I[Flash éxito y redirige a index]
    E --> B
```

Edición y reemplazo de archivos

En `update()`:

- Si llega un archivo nuevo de imagen/modelo, se guarda el nuevo, se persiste la nueva ruta y se **elimina el archivo anterior** del disco `public` solo si empieza por `productos/` y no contiene `..` (guardas contra traversal).
- Si llega el checkbox `quitar_imagen` o `quitar_modelo`, se anula la ruta en BD y se elimina el archivo físico.
- Si no llega ningún archivo nuevo y no se marca para quitar, se conserva el valor actual de BD.
- Los archivos órfanos se limpian en `destroy()`.

Eliminación de producto

- Se eliminan primero los archivos asociados (`imagen_path` y `modelo_3d_path`) del disco `public`.
- Se intenta eliminar el registro del producto.
- Si existen filas en `detalle_pedido` que lo referencian, la FK `restrictOnDelete` impide el borrado; se captura `QueryException` y se redirige con: *"No se puede eliminar: tiene pedidos asociados."* (sin 500, sin borrar el producto ni sus archivos).

Listado

`index()` está paginado, con buscador por nombre/categoría y filtro activo/inactivo. La tabla muestra miniatura (imagen o placeholder), nombre, categoría, precio, stock, activo, indicador de modelo 3D (sí/no) y acciones editar/eliminar (esta última con confirmación).

Gestión de pedidos (RF-06)

Listado

`index()` devuelve los pedidos más recientes primero, paginados, con filtro por estado. Usa *eager loading* `user` y `detallesPedido.producto` para evitar N+1. Cada fila muestra:

- Número de pedido y fecha.
- Cliente: nombre del usuario o **"Invitado"** cuando `user_id = NULL`.
- Total y método de pago legible (Efectivo / Tarjeta / Transferencia).
- Estado como badge con color y control de cambio de estado.

Cambio de estado

`updateEstado(PATCH /admin/pedidos/{pedido}/estado)` valida:

```
$estado in:pendiente,en_preparacion,entregado
```

Solo actualiza el campo `estado` (ningún otro campo es mas-assignable). Los estados `listo` y `cancelado` existen en el enum de base de datos pero **no son seleccionables** desde el panel según el requerimiento RF-06; si existen en registros antiguos, se muestran en su badge correspondiente.

Cada fila tiene una fila expandible (Alpine, accesible con `aria-expanded` y `aria-controls`) que lista las líneas del pedido: producto, cantidad, precio unitario y subtotal.

Diagrama de transiciones de estado

```
stateDiagram-v2
    [*] --> Pendiente
    Pendiente --> EnPreparacion : admin cambia a en_preparacion
    EnPreparacion --> Entregado : admin cambia a entregado
    EnPreparacion --> Pendiente : reversión permitida por el panel
    Pendiente --> Entregado : entrega directa desde pendiente
    Entregado --> [*] : final
```

Los estados `listo` y `cancelado` se muestran en el detalle pero no son seleccionables desde este módulo.

Layout y vistas de administración

Layout `admin.blade.php`

`resources/views/layouts/admin.blade.php` es un layout separado del público (`layouts/app.blade.php`). Características:

- Sidebar lateral con paleta piedra/ámbar (`stone-900` / `stone-800` para el sidebar, `bg-stone-50` para el contenido).
- Marca "Cafetería · Admin", navegación: **Dashboard**, **Productos**, **Pedidos**.
- Enlace "Ver tienda" hacia el catálogo público `/`.
- Barra superior con nombre del usuario autenticado y botón de cierre de sesión.
- Responsive con menú toggle móvil mediante Alpine.js (ya disponible del proyecto).

- Contenido principal con tipografía Instrument Sans y títulos claros.

Vistas creadas

VISTA	RESPONSABILIDAD
`admin/dashboard.blade.php`	Panel con métricas reales: productos activos, pedidos pendientes, pedidos del día y ventas del día (suma de `total` de pedidos no cancelados); accesos rápidos a "Nuevo producto" y "Ver pedidos".
`admin/productos/index.blade.php`	Tabla paginada con buscador y filtro activo/inactivo.
`admin/productos/create.blade.php`	Formulario de nuevo producto.
`admin/productos/edit.blade.php`	Formulario de edición con valores actuales y previsualización de imagen/modelo existente.
`admin/productos/_form.blade.php`	Formulario compartido `multipart/form-data` con `accept="image/*"` para imagen y `accept=".glb,.gltf"` para modelo 3D, errores de validación inline y controles accesibles.
`admin/pedidos/index.blade.php`	Listado de pedidos con filtro por estado, badges de estado y filas expandibles con detalle de líneas.
`components/estado-pedido.blade.php`	Componente de badge de estado reutilizable.

El dashboard y el listado de pedidos evitan consultas N+1 mediante eager loading y agregaciones eficientes.

Cambios en el navbar público

Se añadió el enlace **"Panel de Admin"** visible solo cuando `auth()->user()?->isAdmin()` es verdadero, tanto en la navegación de escritorio como en el menú móvil. Apunta a `admin.dashboard`. Es cosmético; la autorización real está en el middleware.

Pruebas

Suite ejecutada tras el Módulo 4:

COMANDO	RESULTADO
`php artisan test --compact`	**71 tests, 325 aserciones, todos pasando**
`vendor/bin/pint --format agent`	`passed`
`npm run build`	Correcto (aviso preexistente de chunk >500 kB por Three.js)

Se añadieron:

- `tests/Feature/Admin/AdminProductoTest.php` (14 pruebas): invitado redirige, cliente 403, admin 200, create con `UploadedFile::fake()` de imagen y glb, `Storage::fake('public')` verifica archivo guardado y rutas en BD, update reemplaza y borra el anterior, destroy borra producto y archivos, destroy con pedido asociado no rompe (redirect con error), validación de extensión/tamaño.
- `tests/Feature/Admin/AdminPedidoTest.php` (7 pruebas): acceso por rol, listado con filtro, updateEstado válido e inválido, badge/estado visible.
- `tests/Feature/Admin/AdminDashboardTest.php` (5 pruebas): acceso por rol y métricas.

Las 48 pruebas previas de los módulos 1-3 siguen pasando.

Guía de prueba manual: subir un modelo 3D real desde la interfaz

1. **Elevar límites de PHP** (el CLI por defecto tiene `upload_max_filesize=24M` y `post_max_size=32M` tras el ajuste del entorno; si los ves más bajos, relánzalos con `-d upload_max_filesize=24M -d post_max_size=32M`):

   ```
   cd /opt/proyecto_cafeteria
   php artisan serve
   ```

2. **Entrar como administrador**:

EMAIL	CONTRASEÑA
`admin@cafeteria.test`	`password`

   (Estas credenciales se crean con el seeder `php artisan db:seed`.)

3. Ir a **Panel de Admin → Productos → Nuevo producto** (`/admin/productos/create`).

4. Completar nombre, precio, stock, categoría, activo, adjuntar una imagen y seleccionar un archivo `.glb` o `.gltf` de hasta 20 MB. Enviar.

5. Debe aparecer el flash de éxito y el indicador **"Modelo 3D: Sí"** en el listado de productos.

6. Verificar el archivo y la ruta en BD:

   ```
   ls -l storage/app/public/productos
   php artisan tinker --execute 'echo App\Models\Producto::latest("id")->first()->modelo_3d_path;'
   ```

   Deberías ver algo como `productos/abc123.glb`.

7. Abrir la ficha pública `/productos/{id}`: el visor 3D del Módulo 2 cargará el modelo desde `asset('storage/'.$producto->modelo_3d_path)`.

8. Para probar la edición: entrar al producto creado, subir otro `.glb` en sustitución y comprobar que el anterior desapareció de `storage/app/public/productos` y de `modelo_3d_path`.

Recomendaciones de exportación del modelo 3D

- Usar **GLB** (un solo archivo) en lugar de glTF + `.bin` + texturas sueltas, para evitar rutas relativas rotas.
- **Sin compresión Draco ni KTX2/meshopt**: el visor registra `GLTFLoader` sin `DRACOLoader` ni transcoder KTX2.
- Menos de 2–3 MB y unas decenas de miles de triángulos para cumplir el objetivo de carga del Módulo 2 (RNF-02).

Notas importantes

- No se crearon ni modificaron migraciones ni modelos en este módulo; se reutilizaron `productos`, `pedidos`, `detalle_pedido` y sus enums existentes.
- No se implementó inventario ni reportes (Módulo 5).
- El panel es funcional para catálogo y pedidos; los movimientos de inventario esperan el Módulo 5.

06 — Inventario y reportes (Módulo 5)

Este documento describe el módulo de inventario y reportes de ventas. Cubre **RF-07** (inventario de insumos con entradas y salidas) y **RF-08** (reportes de ventas por período y por producto).

El módulo se integra al panel de administración existente (`/admin`, middleware `auth` + `role:admin`) y reutiliza el layout `resources/views/layouts/admin.blade.php` con la paleta piedra/ámbar. No se crearon ni modificaron migraciones ni modelos: se reutilizan `insumos`, `movimientos_inventario`, `pedidos`, `detalle_pedido` y sus enums (`TipoMovimiento`, `EstadoPedido`, `MetodoPago`) creados en módulos previos.

Alcance y requerimientos

REQUERIMIENTO	IMPLEMENTACIÓN ACTUAL
RF-07 — Inventario de insumos	`Admin\AdminInsumoController` (CRUD) y `Admin\AdminMovimientoController` (registro de entrada/salida/ajuste). El stock del insumo se actualiza automáticamente al registrar un movimiento, dentro de una transacción de base de datos.
RF-08 — Reportes de ventas	`Admin\AdminReporteController` con `ventasPorPeriodo` (rango de fechas, total, número de pedidos, evolución diaria) y `ventasPorProducto` (ranking de más vendidos: cantidad y total recaudado).

Seguridad y rutas

Todas las rutas viven dentro del grupo protegido:

```
Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () { ... });
```

MÉTODO	URI	NOMBRE	CONTROLADOR	DESCRIPCIÓN
GET	`/admin/insumos`	`admin.insumos.index`	`Admin\AdminInsumoController`	Listado paginado con buscador y filtro bajo stock
GET	`/admin/insumos/create`	`admin.insumos.create`	`Admin\AdminInsumoController`	Formulario de nuevo insumo
POST	`/admin/insumos`	`admin.insumos.store`	`Admin\AdminInsumoController`	Crear insumo
GET	`/admin/insumos/{insumo}/edit`	`admin.insumos.edit`	`Admin\AdminInsumoController`	Formulario de edición
PUT/PATCH	`/admin/insumos/{insumo}`	`admin.insumos.update`	`Admin\AdminInsumoController`	Actualizar insumo
DELETE	`/admin/insumos/{insumo}`	`admin.insumos.destroy`	`Admin\AdminInsumoController`	Eliminar (bloqueado si tiene movimientos)
GET	`/admin/insumos/{insumo}/movimientos/create`	`admin.insumos.movimientos.create`	`Admin\AdminMovimientoController`	Formulario de nuevo movimiento
POST	`/admin/insumos/{insumo}/movimientos`	`admin.insumos.movimientos.store`	`Admin\AdminMovimientoController`	Registrar movimiento y actualizar stock

GET	`/admin/reportes/ventas-periodo`	`admin.reportes.ventasPeriodo`	`Admin\AdminReporteController`	Reporte de ventas por rango de fechas
GET	`/admin/reportes/ventas-producto`	`admin.reportes.ventasProducto`	`Admin\AdminReporteController`	Ranking de productos más vendidos

Comportamiento por sesión

- **Invitado**: redirección (302) a `/login` en cualquier ruta `/admin`.
- **Cliente autenticado**: `403` al acceder a cualquier ruta del panel.
- **Administrador autenticado**: acceso completo.

Inventario (RF-07)

CRUD de insumos

`AdminInsumoController` implementa `index`, `create`, `store`, `edit`, `update`, `destroy`.

Validación (`StoreInsumoRequest` / `UpdateInsumoRequest`):

CAMPO	REGLAS
`nombre`	`required
`unidad_medida`	`required
`stock_actual`	`required
`stock_minimo`	`required
`costo_unitario`	`nullable

Registro de movimientos y actualización automática de stock

`AdminMovimientoController@store` registra un movimiento y actualiza el stock del insumo de forma atómica:

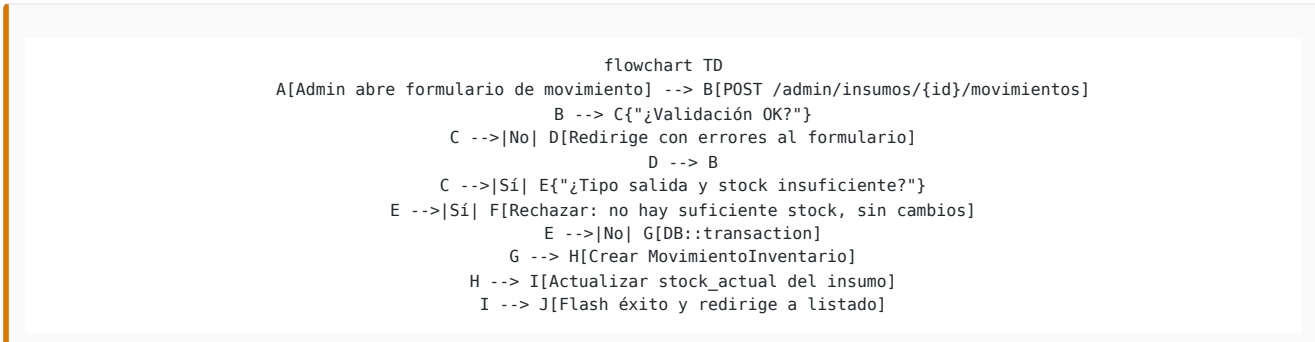
Validación (`StoreMovimientoRequest`):

CAMPO	REGLAS
`tipo`	`required
`cantidad`	`required
`motivo`	`nullable

Lógica transaccional (`DB::transaction`):

1. Si el tipo es `salida`, se valida que `stock_actual - cantidad >= 0`. Si no, se rechaza con error de validación y **ningún cambio** se persiste (rollback implícito al no entrar a la transacción).
2. Dentro de la transacción:
   - `entrada`: `stock_actual = stock_actual + cantidad`
   - `salida`: `stock_actual = stock_actual - cantidad`
   - `ajuste`: `stock_actual = cantidad` (se establece el stock físico contado)
3. Se crea el registro en `movimientos_inventario` con `insumo_id`, `tipo`, `cantidad`, `motivo` y `user_id` del administrador autenticado.
4. Se actualiza `stock_actual` del insumo.

Diagrama de flujo del registro de movimiento:



Eliminación protegida

Un insumo que tenga movimientos asociados no se puede eliminar (FK `restrictOnDelete`). Se captura `QueryException` y se redirige con: *"No se puede eliminar: tiene movimientos de inventario asociados."*

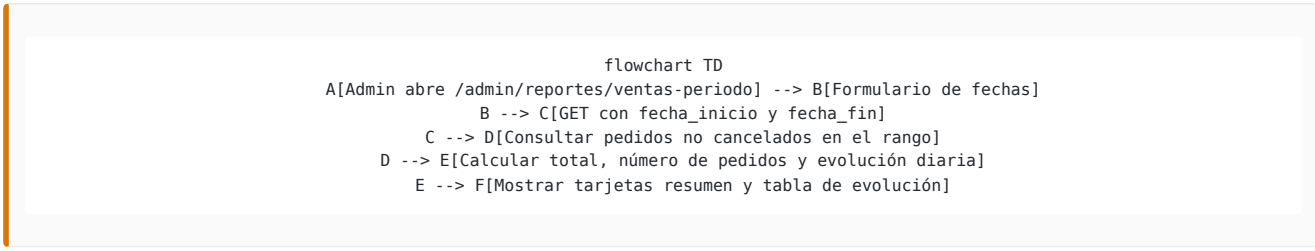
Reportes (RF-08)

Ventas por período (`ventasPorPeriodo`)

Recibe `fecha_inicio` y `fecha_fin` (query params, opcionales; por defecto los últimos 30 días). Valida que `fecha_fin >= fecha_inicio`.

Consulta pedidos no cancelados (`estado <> 'cancelado'`) con `created_at` entre las fechas, y devuelve:

- **Total vendido**: suma de `total`.
- **Número de pedidos**: conteo de pedidos.
- **Ticket promedio**: total dividido por número de pedidos.
- **Evolución diaria**: agrupación por `DATE(created_at)` con pedidos y total por día, mostrada como tabla con barras de progreso proporcionales.



Ventas por producto (`ventasPorProducto`)

Ranking de los productos más vendidos. Agrupa `detalle_pedido` por `producto_id`, suma `cantidad` y `subtotal`, hace join con `productos` para obtener el nombre, y ordena por cantidad descendente (top 10). Solo considera pedidos no cancelados.

Muestra tabla con: producto, cantidad vendida, total recaudado y participación porcentual sobre el total general.

Vistas y layout

Se reutiliza `resources/views/layouts/admin.blade.php` (sidebar con paleta piedra/ámbar, navbar superior, `<x-flash-banner />`).

Vistas creadas

VISTA	ARCHIVO	RESPONSABILIDAD
Insumos listado	`resources/views/admin/insumos/index.blade.php`	Tabla con stock, badge de bajo stock, buscador, filtro
Insumo crear/editar	`create.blade.php`, `edit.blade.php`, `_form.blade.php`	Formulario con validación inline

Movimiento crear	`resources/views/admin/insumos/movimientos/create.blade.php`	Formulario tipo entrada/salida/ajuste con cantidad y motivo
Reporte por período	`resources/views/admin/reportes/ventas-periodo.blade.php`	Filtro de fechas, tarjetas resumen, tabla evolución
Reporte por producto	`resources/views/admin/reportes/ventas-producto.blade.php`	Tabla ranking con barras de participación

Sidebar

Se agregaron dos enlaces al sidebar del layout admin:

- **Inventario** → `admin.insumos.index`
- **Reportes** → `admin.reportes.ventasPeriodo`

Están resaltados según la ruta activa y siguen la misma paleta stone/ámbar que los enlaces existentes (Dashboard, Productos, Pedidos).

Seeder de insumos

Se creó `database/seeders/InsumoSeeder` (idempotente con `updateOrCreate`) y se registró en `DatabaseSeeder`. Crea 3 insumos de prueba:

INSUMO	UNIDAD	STOCK INICIAL	STOCK MÍNIMO	COSTO UNITARIO
Café en grano	kg	10.000	2.000	S/ 18.50
Leche	lt	20.000	5.000	S/ 2.80
Vasos	unidad	100.000	20.000	S/ 0.15

Ejecutar con: `php artisan db:seed --force`.

Pruebas

Suite ejecutada tras el Módulo 5:

COMANDO	RESULTADO
`php artisan test`	**81 tests, 354 aserciones, todos pasando**
`vendor/bin/pint --format agent`	`fixed` (estilo aplicado)
`npm run build`	Correcto (aviso preexistente de chunk >500 kB por Three.js)

Se añadieron:

- `tests/Feature/Admin/AdminInsumoTest.php` (7 pruebas): acceso por rol, listado, creación, validación de stock negativo, actualización, eliminación bloqueada con movimientos.
- `tests/Feature/Admin/AdminMovimientoTest.php` (5 pruebas): entrada aumenta stock, salida disminuye stock, salida excedente rechaza sin cambios, ajuste establece stock, invitado redirige.
- `tests/Feature/Admin/AdminReporteTest.php` (6 pruebas): acceso por rol, filtro de fechas, ranking de productos, invitado/cliente rechazados.

Los 71 tests previos (módulos 1-4) siguen pasando.

Cómo probar desde la interfaz

1. Levantar el servidor (con límites de PHP para subidas, aunque el inventario no sube archivos):

   ```
   cd /opt/proyecto_cafeteria
   php artisan serve
   ```

2. Entrar como administrador:

EMAIL	CONTRASEÑA

`admin@cafeteria.test`	`password`

3. Ir a **Panel de Admin** → **Inventario**. Verá los 3 insumos del seeder (Café en grano, Leche, Vasos) con su stock y badge de bajo stock si aplica.
4. Para registrar un movimiento: hacer clic en "Movimiento" de un insumo, elegir tipo (Entrada/Salida/Ajuste), cantidad y motivo. Al guardar, el stock del insumo se actualiza automáticamente y se muestra un flash de éxito.
5. Probar una salida que exceda el stock: debe rechazarse con un error y el stock no debe cambiar.
6. Ir a **Panel de Admin** → **Reportes** → **Ventas por período** o **Ventas por producto** para ver los reportes. Si no hay pedidos, se muestra un estado vacío elegante.

Notas importantes

- No se crearon ni modificaron migraciones ni modelos en este módulo. Se reutilizó el esquema existente.
- El stock se actualiza dentro de `DB::transaction` para evitar inconsistencias.
- Las salidas se validan contra el stock disponible antes de la transacción, evitando sobreventa.
- Los estados de pedido `listo` y `cancelado` se excluyen de los reportes de ventas (solo se consideran no cancelados).
- No se implementaron movimientos de inventario vinculados a pedidos de venta automáticamente; el registro de movimientos es una acción administrativa manual.

07 — Base de datos (Especificación técnica completa)

Documento de especificación del esquema relacional del sistema de ventas para cafetería. Todas las tablas viven en la base de datos `cafeteria` (MariaDB 11.8, charset `utf8mb4`) y se crean mediante las migraciones de `database/migrations`.

2.3.1 Modelo Entidad-Relación

El siguiente diagrama `erDiagram` describe las entidades del dominio, sus atributos y las relaciones entre ellas.

```
                    erDiagram
          users ||--o{ pedidos : realiza
    users |o--o{ movimientos_inventario : audita
      pedidos ||--o{ detalle_pedido : contiene
    productos ||--o{ detalle_pedido : referencia
insumos ||--o{ movimientos_inventario : registra

                      users {
                     bigint id PK
                   varchar name UK1
                   varchar email UK
                timestamp email_verified_at
                   varchar password
                     varchar role
                varchar remember_token
                 timestamp created_at
                 timestamp updated_at
                        }

                   productos {
                     bigint id PK
                    varchar nombre
                   text descripcion
                    decimal precio
                  int unsigned stock
                   varchar categoria
                  varchar imagen_path
                varchar modelo_3d_path
                    boolean activo
                 timestamp created_at
                 timestamp updated_at
                        }

                    pedidos {
                     bigint id PK
                   bigint user_id FK
                     varchar estado
                  varchar metodo_pago
                     decimal total
                       text notas
                 timestamp created_at
                 timestamp updated_at
                        }

                 detalle_pedido {
                     bigint id PK
                  bigint pedido_id FK
                bigint producto_id FK
                int unsigned cantidad
               decimal precio_unitario
                    decimal subtotal
                 timestamp created_at
                 timestamp updated_at
                        }

                    insumos {
```

```
        bigint id PK
      varchar nombre
  varchar unidad_medida
   decimal stock_actual
   decimal stock_minimo
  decimal costo_unitario
   timestamp created_at
   timestamp updated_at
          }

movimientos_inventario {
          bigint id PK
       bigint insumo_id FK
          varchar tipo
        decimal cantidad
         varchar motivo
       bigint user_id FK
      timestamp created_at
      timestamp updated_at
          }
```

Cardinalidades

RELACIÓN	CARDINALIDAD	DESCRIPCIÓN
`users` → `pedidos`	1:N (0:N)	Un usuario realiza cero o muchos pedidos. Los pedidos de invitado tienen `user_id = NULL`.
`pedidos` → `users`	N:1 (0:1)	Cada pedido pertenece a cero o un usuario. `RESTRICT` al eliminar.
`pedidos` → `detalle_pedido`	1:N	Un pedido agrupa cero o muchas líneas de detalle.
`detalle_pedido` → `pedidos`	N:1	Cada línea pertenece a un solo pedido. `CASCADE` al eliminar el pedido.
`productos` → `detalle_pedido`	1:N	Un producto puede aparecer en cero o muchas líneas.
`detalle_pedido` → `productos`	N:1	Cada línea referencia un solo producto. `RESTRICT` al eliminar.
`insumos` → `movimientos_inventario`	1:N	Un insumo acumula cero o muchos movimientos.
`movimientos_inventario` → `insumos`	N:1	Cada movimiento afecta a un solo insumo. `RESTRICT` al eliminar.
`users` → `movimientos_inventario`	1:N (0:N)	Un usuario puede auditar cero o muchos movimientos.
`movimientos_inventario` → `users`	N:1 (0:1)	`user_id` es opcional (`SET NULL`); el histórico sobrevive al usuario.

2.3.2 Modelo Relacional

A continuación se detalla cada tabla con sus columnas, tipos, restricciones y claves foráneas, junto con las cardinalidades de las relaciones.

Tabla `users`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`name`	`varchar(255)`	NO		Nombre visible del usuario.
`email`	`varchar(255)`	NO	UK	Credencial de acceso, único.
`email_verified_at`	`timestamp`	SÍ		Fecha de verificación de correo.
`password`	`varchar(255)`	NO		Hash de la contraseña (cast `hashed`).
`role`	`varchar(255)`	NO	IK	Rol (`cliente` / `admin`), default `'cliente'`.
`remember_token`	`varchar(100)`	SÍ		Token "recordarme" de Laravel.
`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Índice: `users_role_index` sobre `role`. Clave foránea: ninguna propia (es referenciada por otras tablas).

Tabla `productos`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`nombre`	`varchar(255)`	NO		Nombre del producto.
`descripcion`	`text`	SÍ		Descripción libre del producto.
`precio`	`decimal(10,2)`	NO		Precio de venta.
`stock`	`int unsigned`	NO		Stock en unidades (sin negativos).
`categoria`	`varchar(255)`	SÍ		Categoría libre (`bebidas`, `postres`, `comida`).
`imagen_path`	`varchar(255)`	SÍ		Ruta relativa a `storage/app/public`.
`modelo_3d_path`	`varchar(255)`	SÍ		Ruta relativa al modelo GLB/GLTF.
`activo`	`tinyint(1)`	NO		Visible en catálogo; default `true`.

`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Índice compuesto: `productos_activo_categoria_index` sobre `(activo, categoria)`.

Tabla `pedidos`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`user_id`	`bigint unsigned`	SÍ	FK → `users.id`	Autor del pedido; `NULL` en pedidos de invitado.
`estado`	`varchar(255)`	NO	IK	Estado del pedido (default `'pendiente'`).
`metodo_pago`	`varchar(255)`	NO		Método de pago (default `'efectivo'`).
`total`	`decimal(10,2)`	NO		Importe total consolidado.
`notas`	`text`	SÍ		Observaciones del cliente.
`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Clave foránea: `pedidos.user_id` → `users.id` (`ON DELETE RESTRICT`). Índice: `pedidos_estado_index` sobre `estado`.

Tabla `detalle_pedido`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`pedido_id`	`bigint unsigned`	NO	FK → `pedidos.id`	Pedido al que pertenece la línea.
`producto_id`	`bigint unsigned`	NO	FK → `productos.id`	Producto de la línea.
`cantidad`	`int unsigned`	NO		Unidades del producto.
`precio_unitario`	`decimal(10,2)`	NO		Precio snapshot al momento de la compra.
`subtotal`	`decimal(10,2)`	NO		`cantidad × precio_unitario`.
`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Claves foráneas: `detalle_pedido.pedido_id` → `pedidos.id` (`ON DELETE CASCADE`); `detalle_pedido.producto_id` → `productos.id` (`ON DELETE RESTRICT`).

Tabla `insumos`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`nombre`	`varchar(255)`	NO		Nombre del insumo.
`unidad_medida`	`varchar(255)`	NO		Unidad (`gramos`, `mililitros`, `unidades`).
`stock_actual`	`decimal(10,3)`	NO		Stock actual con 3 decimales.
`stock_minimo`	`decimal(10,3)`	NO		Umbral de reposición.
`costo_unitario`	`decimal(10,2)`	SÍ		Costo unitario del insumo.
`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Tabla `movimientos_inventario`

CAMPO	TIPO	NULO	CLAVE	DESCRIPCIÓN
`id`	`bigint unsigned`	NO	PK	Identificador autoincremental.
`insumo_id`	`bigint unsigned`	NO	FK → `insumos.id`	Insumo afectado por el movimiento.
`tipo`	`varchar(255)`	NO		Tipo (`entrada`, `salida`, `ajuste`).
`cantidad`	`decimal(10,3)`	NO		Magnitud del movimiento.
`motivo`	`varchar(255)`	SÍ		Justificación opcional.
`user_id`	`bigint unsigned`	SÍ	FK → `users.id`	Responsable del registro; `SET NULL`.
`created_at`	`timestamp`	SÍ		Marca de tiempo de creación.
`updated_at`	`timestamp`	SÍ		Marca de tiempo de actualización.

Claves foráneas: `movimientos_inventario.insumo_id` → `insumos.id` (`ON DELETE RESTRICT`); `movimientos_inventario.user_id` → `users.id` (`ON DELETE SET NULL`). Índice compuesto: `(insumo_id, created_at)`.

Tablas de infraestructura de Laravel

Además de las tablas de negocio, las migraciones base crean tablas de soporte del framework:

TABLA	PROPÓSITO
`password_reset_tokens`	Tokens de recuperación de contraseña (Breeze).
`sessions`	Almacenamiento de sesiones (`SESSION_DRIVER=database`).
`cache` / `cache_locks`	Caché y bloqueos (`CACHE_STORE=database`).
`jobs` / `job_batches` / `failed_jobs`	Cola de trabajos (`QUEUE_CONNECTION=database`).

2.3.3 Normalización

El esquema cumple con las tres primeras formas normales (1FN, 2FN y 3FN). A continuación se justifica cada una.

1FN — Primera Forma Normal

Una relación está en 1FN cuando todos sus dominios son atómicos (no contiene valores multivalor ni grupos repetidos) y cada tupla es única.

- Todas las columnas almacenan valores atómicos: no hay columnas con listas, conjuntos ni estructuras anidadas. Campos como `detalle_pedido` no replican datos del pedido ni del producto; cada fila es una línea independiente.
- Cada tabla tiene una clave primaria (`id` , `bigint unsigned` autoincremental) que garantiza la unicidad de cada tupla.
- No existen grupos repetidos ni columnas numeradas (por ejemplo, no hay `producto_1` , `producto_2` , etc.).

Conclusión: Todas las relaciones del esquema están en 1FN.

2FN — Segunda Forma Normal

Una relación está en 2FN cuando está en 1FN y cada atributo no clave depende totalmente de toda la clave primaria (no hay dependencias parciales de una parte de una clave compuesta).

- Todas las claves primarias son simples (una sola columna `id`), por lo que no puede existir dependencia parcial de una parte de una clave compuesta.
- Cada atributo no clave de cada tabla depende de la totalidad de su clave primaria (`id`).

Conclusión: Al no existir claves compuestas, el esquema cumple trivialmente con 2FN.

3FN — Tercera Forma Normal

Una relación está en 3FN cuando está en 2FN y no existen dependencias transitivas entre atributos no clave (ningún atributo no clave depende de otro atributo no clave).

- En `users`, `role`, `name`, `email`, etc. dependen directamente de `id` (la clave), no entre sí.
- En `productos`, `nombre`, `precio`, `stock`, `categoria`, etc. dependen directamente de `id`.
- En `pedidos`, `total`, `estado`, `metodo_pago`, `user_id` dependen directamente de `id`.
- En `detalle_pedido`, `subtotal` depende de `cantidad` y `precio_unitario`, pero estos últimos son atributos de la propia línea (parte de la clave de la relación de hecho), y el valor se guarda como snapshot histórico; no hay una dependencia transitiva que infrinja 3FN porque `subtotal` es un atributo derivado almacenado a propósito para conservar el histórico del importe pagado.
- En `insumos` y `movimientos_inventario`, todos los atributos no clave dependen directamente de su respectiva `id`.

Conclusión: No hay dependencias transitivas entre atributos no clave; el esquema cumple con 3FN.

Resumen: El esquema relacional está en 1FN, 2FN y 3FN.

2.3.4 Modelo Físico

A continuación se extrae el código SQL equivalente a las migraciones de Laravel que crean el esquema de negocio. El motor es InnoDB con charset `utf8mb4` y collation `utf8mb4_unicode_ci`.

2.3.4.1 Script SQL de las tablas de negocio

```sql
-- ============================================
-- Tabla: users (base de autenticación de Laravel)
-- ============================================
CREATE TABLE `users` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `name` varchar(255) NOT NULL,
    `email` varchar(255) NOT NULL,
    `email_verified_at` timestamp NULL DEFAULT NULL,
    `password` varchar(255) NOT NULL,
    `remember_token` varchar(100) NULL DEFAULT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`),
    UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Columna role (migración 2026_11_08_160119_add_role_to_users_table)
ALTER TABLE `users`
    ADD `role` varchar(255) NOT NULL DEFAULT 'cliente' AFTER `password`,
    ADD INDEX `users_role_index` (`role`);

-- ============================================
-- Tabla: productos
-- ============================================
CREATE TABLE `productos` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `nombre` varchar(255) NOT NULL,
    `descripcion` text NULL DEFAULT NULL,
    `precio` decimal(10,2) NOT NULL,
    `stock` int unsigned NOT NULL DEFAULT 0,
    `categoria` varchar(255) NULL DEFAULT NULL,
    `imagen_path` varchar(255) NULL DEFAULT NULL,
    `modelo_3d_path` varchar(255) NULL DEFAULT NULL,
    `activo` tinyint(1) NOT NULL DEFAULT 1,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`),
    INDEX `productos_activo_categoria_index` (`activo`, `categoria`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================
-- Tabla: pedidos
-- ============================================
CREATE TABLE `pedidos` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `user_id` bigint unsigned NOT NULL,
    `estado` varchar(255) NOT NULL DEFAULT 'pendiente',
    `total` decimal(10,2) NOT NULL DEFAULT 0,
    `notas` text NULL DEFAULT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`),
    INDEX `pedidos_estado_index` (`estado`),
    CONSTRAINT `pedidos_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Modificación: user_id nullable + metodo_pago (migración 2026_11_08_163735)
ALTER TABLE `pedidos`
    MODIFY `user_id` bigint unsigned NULL DEFAULT NULL,
    ADD `metodo_pago` varchar(255) NOT NULL DEFAULT 'efectivo' AFTER `estado`;

-- ============================================
-- Tabla: detalle_pedido
-- ============================================
CREATE TABLE `detalle_pedido` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `pedido_id` bigint unsigned NOT NULL,
    `producto_id` bigint unsigned NOT NULL,
    `cantidad` int unsigned NOT NULL,
    `precio_unitario` decimal(10,2) NOT NULL,
    `subtotal` decimal(10,2) NOT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`),
    CONSTRAINT `detalle_pedido_pedido_id_foreign` FOREIGN KEY (`pedido_id`) REFERENCES `ped
    CONSTRAINT `detalle_pedido_producto_id_foreign` FOREIGN KEY (`producto_id`) REFERENCES
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================
-- Tabla: insumos
-- ============================================
CREATE TABLE `insumos` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `nombre` varchar(255) NOT NULL,
    `unidad_medida` varchar(255) NOT NULL,
    `stock_actual` decimal(10,3) NOT NULL DEFAULT 0,
    `stock_minimo` decimal(10,3) NOT NULL DEFAULT 0,
    `costo_unitario` decimal(10,2) NULL DEFAULT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ============================================
-- Tabla: movimientos_inventario
-- ============================================
CREATE TABLE `movimientos_inventario` (
    `id` bigint unsigned NOT NULL AUTO_INCREMENT,
    `insumo_id` bigint unsigned NOT NULL,
    `tipo` varchar(255) NOT NULL,
    `cantidad` decimal(10,3) NOT NULL,
    `motivo` varchar(255) NULL DEFAULT NULL,
    `user_id` bigint unsigned NULL DEFAULT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`id`),
    INDEX `movimientos_inventario_insumo_id_created_at_index` (`insumo_id`, `created_at`),
    CONSTRAINT `movimientos_inventario_insumo_id_foreign` FOREIGN KEY (`insumo_id`) REFEREN
    CONSTRAINT `movimientos_inventario_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

2.3.4.2 Diccionario de Datos

A continuación se presenta el diccionario de datos consolidado de todas las tablas de negocio.

CAMPO	TABLA	TIPO DE DATO	NULO	DESCRIPCIÓN
`id`	users	`bigint unsigned`	NO	Clave primaria autoincremental.
`name`	users	`varchar(255)`	NO	Nombre visible del usuario.
`email`	users	`varchar(255)`	NO	Correo electrónico, credencial de acceso (único).
`email_verified_at`	users	`timestamp`	SÍ	Fecha y hora de verificación del correo.
`password`	users	`varchar(255)`	NO	Hash de la contraseña (cast `hashed`).
`role`	users	`varchar(255)`	NO	Rol del usuario (`cliente`, `admin`); default `'cliente'`.
`remember_token`	users	`varchar(100)`	SÍ	Token de "recordarme" de Laravel.
`created_at`	users	`timestamp`	SÍ	Marca de tiempo de creación del registro.
`updated_at`	users	`timestamp`	SÍ	Marca de tiempo de última actualización.
`id`	productos	`bigint unsigned`	NO	Clave primaria autoincremental.
`nombre`	productos	`varchar(255)`	NO	Nombre del producto.
`descripcion`	productos	`text`	SÍ	Descripción libre del producto.
`precio`	productos	`decimal(10,2)`	NO	Precio de venta en moneda local.
`stock`	productos	`int unsigned`	NO	Cantidad en unidades disponibles.
`categoria`	productos	`varchar(255)`	SÍ	Categoría del producto.
`imagen_path`	productos	`varchar(255)`	SÍ	Ruta relativa a `storage/app/public` de la imagen.
`modelo_3d_path`	productos	`varchar(255)`	SÍ	Ruta relativa al modelo 3D (GLB/GLTF).
`activo`	productos	`tinyint(1)`	NO	Indica si el producto es visible en el catálogo.
`created_at`	productos	`timestamp`	SÍ	Marca de tiempo de creación.
`updated_at`	productos	`timestamp`	SÍ	Marca de tiempo de última actualización.

`id`	pedidos	`bigint unsigned`	NO	Clave primaria autoincremental.
`user_id`	pedidos	`bigint unsigned`	SÍ	FK → `users.id`; `NULL` para pedidos de invitado.
`estado`	pedidos	`varchar(255)`	NO	Estado del pedido (default `'pendiente'`).
`metodo_pago`	pedidos	`varchar(255)`	NO	Método de pago (default `'efectivo'`).
`total`	pedidos	`decimal(10,2)`	NO	Importe total consolidado del pedido.
`notas`	pedidos	`text`	SÍ	Observaciones del cliente.
`created_at`	pedidos	`timestamp`	SÍ	Marca de tiempo de creación.
`updated_at`	pedidos	`timestamp`	SÍ	Marca de tiempo de última actualización.
`id`	detalle_pedido	`bigint unsigned`	NO	Clave primaria autoincremental.
`pedido_id`	detalle_pedido	`bigint unsigned`	NO	FK → `pedidos.id` (CASCADE).
`producto_id`	detalle_pedido	`bigint unsigned`	NO	FK → `productos.id` (RESTRICT).
`cantidad`	detalle_pedido	`int unsigned`	NO	Unidades del producto en la línea.
`precio_unitario`	detalle_pedido	`decimal(10,2)`	NO	Precio snapshot al momento de la compra.
`subtotal`	detalle_pedido	`decimal(10,2)`	NO	`cantidad × precio_unitario`.
`created_at`	detalle_pedido	`timestamp`	SÍ	Marca de tiempo de creación.
`updated_at`	detalle_pedido	`timestamp`	SÍ	Marca de tiempo de última actualización.
`id`	insumos	`bigint unsigned`	NO	Clave primaria autoincremental.
`nombre`	insumos	`varchar(255)`	NO	Nombre del insumo.
`unidad_medida`	insumos	`varchar(255)`	NO	Unidad de medida (gramos, mililitros, unidades).
`stock_actual`	insumos	`decimal(10,3)`	NO	Stock actual con tres decimales.
`stock_minimo`	insumos	`decimal(10,3)`	NO	Umbral mínimo de reposición.
`costo_unitario`	insumos	`decimal(10,2)`	SÍ	Costo unitario del insumo.

`created_at`	insumos	`timestamp`	SÍ	Marca de tiempo de creación.
`updated_at`	insumos	`timestamp`	SÍ	Marca de tiempo de última actualización.
`id`	movimientos_inventario	`bigint unsigned`	NO	Clave primaria autoincremental.
`insumo_id`	movimientos_inventario	`bigint unsigned`	NO	FK → `insumos.id` (RESTRICT).
`tipo`	movimientos_inventario	`varchar(255)`	NO	Tipo de movimiento (`entrada` , `salida` , `ajuste`).
`cantidad`	movimientos_inventario	`decimal(10,3)`	NO	Magnitud del movimiento.
`motivo`	movimientos_inventario	`varchar(255)`	SÍ	Justificación opcional del movimiento.
`user_id`	movimientos_inventario	`bigint unsigned`	SÍ	FK → `users.id` (SET NULL); responsable del registro.
`created_at`	movimientos_inventario	`timestamp`	SÍ	Marca de tiempo de creación.
`updated_at`	movimientos_inventario	`timestamp`	SÍ	Marca de tiempo de última actualización.

Referencias cruzadas

- 01-base-de-datos.md — Documento de referencia del esquema relacional, diccionario de tablas ampliado y relaciones Eloquent.
- 02-roles-y-autenticacion.md — Autenticación, roles y protección del panel `/admin` .
- 03-catalogo-y-visor-3d.md — Catálogo público, ficha de producto y visor Three.js.
- 04-carrito-y-pedidos.md — Carrito de sesión, checkout transaccional y pedidos.
- 05-administracion.md — Panel de administración, CRUD de productos y gestión de pedidos.
- 06-inventario-y-reportes.md — Inventario de insumos y reportes de ventas.

CAFÉ DE ESPECIALIDAD

Nuestro catálogo

Granos seleccionados, preparaciones cuidadas y una experiencia pensada para disfrutar sin prisa. Explora cada bebida y descubre su detalle.

CAFÉ · Disponible

Capuchino

Nuestro espresso con leche cremosa y una corona generosa de espuma. Un clásico italiano servido e...

S/ 3.60 Ver detalle ›

CAFÉ · Disponible

Espresso

Un disparo concentrado de nuestro blend de la casa: cuerpo intenso, notas de cacao amargo y un...

S/ 2.50 Ver detalle ›

CAFÉ · Disponible

Latte

Espresso de especialidad combinado con leche vaporizada sedosa y una fina capa de...

S/ 3.80 Ver detalle ›

Volver al catálogo



Modelo 3D no disponible

Estamos preparando una vista tridimensional de este producto. Mientras tanto, descubre todos sus detalles en esta ficha.

CAFÉ

Espresso

Un disparo concentrado de nuestro blend de la casa: cuerpo intenso, notas de cacao amargo y un final limpio que invita al siguiente sorbo.

S/ 2.50

Disponible 40 unidads en stock

CANTIDAD

1

Agregar al carrito

Puedes agregar hasta 10 unidads por pedido.

RESUMEN DE LA CAFETERÍA

Hola, Administrador Cafetería

Gestiona el catálogo, revisa los pedidos del día y mantén la operación al día desde un solo lugar.

+ Nuevo producto | Ver pedidos

Productos activos	Pedidos pendientes	Pedidos de hoy	Ventas de hoy
3	0	0	S/ 0.00
Disponibles en el catálogo público	Esperando preparación	Registrados durante el día	Pedidos no cancelados de hoy

Últimos pedidos

Los movimientos más recientes de la cafetería.

Ver todos

Todavía no hay pedidos registrados.

Inventario de insumos

4 insumos registrados.

+ Nuevo insumo

Buscar

Nombre o unidad

Filtro

Todos

Filtrar

stock_actual <= $insumo->stock_minimo; stock_actual <= $insumo->stock_minimo; stock_actual <= $insumo->stock_minimo; stock_actual <= $insumo->stock_minimo;

INSUMO	UNIDAD	STOCK ACTUAL	STOCK MÍNIMO	COSTO UNIT.	ESTADO	ACCIONES
Vasos 0 movimientos registrados	unidad	100.000	20.000	S/ 0.15	OK	Editar Movimiento Eliminar
Leche 0 movimientos registrados	lt	20.000	5.000	S/ 2.80	OK	Editar Movimiento Eliminar
Café en grano 0 movimientos registrados	kg	10.000	2.000	S/ 18.50	OK	Editar Movimiento Eliminar
Test Directo 0 movimientos registrados	u	1.000	0.000	—	OK	Editar Movimiento Eliminar

Ventas por período

Evolución de ventas y totales en el rango seleccionado.

← Ranking por producto

Desde 10/10/2026

Hasta 08/11/2026

Filtrar

TOTAL VENDIDO
S/ 0.00

NÚMERO DE PEDIDOS
0

TICKET PROMEDIO
S/ 0.00

Evolución diaria

No hay ventas en el rango seleccionado.